ANCHOR UNIVERSITY
LAGOS

AN INTELLIGENT SHELF-LIFE FOOD ESTIMATION SYSTEM

FRESCO: A Fruit Freshness and Shelf-Life Estimation Platform

BY

JEREMIAH SHADAMORO OLUWATOSIN

MATRIC NUMBER: AUL/CMP/22/082

POSSIBLE OLOFU ADA'INAKA

MATRIC NUMBER: AUL/IFT/22/009

Supervised by:

Dr. Babalola Asegunloluwa

A Project submitted to the Department of Computing in partial fulfilment of the requirements for the award of Bachelor of Science (B.Sc.) Degree in Computer Science

August, 2026

Table of Contents

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Food waste is an alarming global crisis. A recent report states that nearly one-third of all food produced for human consumption worldwide is lost or wasted each year, equating to about 1.3 billion tonnes of food with an economic cost nearing $940 billion dollars each year (FAO, 2019). A significant proportion of food waste is reported to occur in the home.

Consumers often throw away food which may not have gone bad, but they have not been able to determine the correct state of consumption for the food.

The dates on packaging are the most common indicator of food condition, but were developed for an ideally monitored environment that is unattainable in the home (Leena, 2024). Food is subject to handling by several parties along its journey. Many factors affect the state of preservation such as temperatures during transit and storage, and the way food is handled. A print-out date does not take these varying factors into account.

In particular, this presents a problem with fresh produce like fruit. Most fruit comes to market not with an expiration date but is visually judged by the consumer as to freshness. Factors such as looking at and smelling fruit are unreliable and some fruits carry such conditions such as mold before a consumer detects a problem. A piece of fruit may already be spoiled from the day before if mould growth has started (Leena, 2024).

Likewise an unspoilt fruit may be thrown away and this would also constitute food waste.

The research presented here seeks to overcome this issue by using the latest developments in computer vision to detect the freshness of fruits by photographic methods (Fu et al., 2022). Given most people now use a mobile phone, their camera is

excellent enough to capture these details and produce some means by which food waste can be reduced. A system which could scan a picture taken by a smart phone camera of a fruit to identify this, estimate how long it would continue to be fresh and store advice about fruit, including advising of any stored items known to adversely affect certain fruits stored within the same vicinity is suggested here. Fresco is an application which aims to achieve exactly this.

1.2 Problem Statement

The primary research problems are:

1. Determining freshness: Determining freshness of fresh fruit remains a crucial but difficult challenge for households. Most users judge it based on appearance, touch and smell, which are often subjective and may vary from person to person. Scientific evidence suggests that when mold is visually evident on a fruit's surface, the internal contamination may have been developing for a day or two(Tournas, 2005).

Thus fruit is thrown out prematurely while it still remains good to consume, or else the fruit is consumed after decay has already occurred.

This creates a necessity to eliminate subjectivity from this problem so as to render objective assessment.

2. Ethylene Interaction Knowledge: Consumers are largely unaware of storage reactions involving ethylene gas. Since some produce reacts poorly with other fruits by creating ethylene gas, a gas that causes ripening and spoiling of produce; like strawberries could be preserved for up to three days when kept alongside bananas. While this information is usually known to all industrial fruit marketers it is unknown to the everyday consumer so unconsciously deteriorates their food produce due to inappropriate storage combinations.

3. Inaccessibility of existing solutions: Many studies were performed involving the use of computer vision, gas sensors and temperature monitoring. Research obtained reliability of over

94% using gas sensors and temperature monitoring (Wang et al., 2025) and the possible uses of hyperspectral imaging to detect internal bruising (Patel et al., 2024). All these studies were developed using consumer-unfriendly, and expensive machines; hence these applications were only used within a laboratory or industrial environment rather than by consumer for everyday use.

1.3 Aim of the Study:

The aim of this study is to design, develop, and implement Fresco-a web application leveraging computer vision on a smartphone that, upon taking a picture: identifies fruits, estimates freshness by visual attributes, predicts remaining shelf-life based on storage condition inputs, flags ethylene compatibility issues between fruits and gives recommended storage advice. The implementation will require no specialized hardware except a smartphone's existing camera.

1.4 Study Objectives:

To fulfill the research aim, this study focuses on the following four objectives:

1. To create an automated fruit identification mechanism that identifies common fruit varieties from the taken photograph, using MobileNetV2 transfer learning, without manual intervention.

2. To generate an objective continuous fresh quality score (0.0 - 1.0) based on a visual attribute estimation that the score can then be mapped to 5 freshness stages (fresh, slightly aged, ageing, old, spoiled).

3. To estimate personal remaining days, built upon fruit identity, freshness level and entered storage method; this calculation relies on an organized dataset of fruit shelf-lives.

1.5 Research Questions:

The questions that need to be answered throughout this research are:

* How can the specific fruit varieties and their fresh quality stage (age) from the image from a consumer smartphone's camera be identified through use of CNN trained via a smartphone accessible framework and architecture?
* How can visual data from a fruit's image be mapped using a regression model, so as to obtain a

continuous freshness attribute value?

* When combining the value of the obtained freshness attribute, and chosen storage methods, how can we generate shelf life estimates for different produce varieties; that remain reliable?

* How to apply acquired knowledge about ethylene interaction when preparing user-based applications to warn consumers of incompatible fruits stored together?

* What level of accuracy, practicality and use of smartphone based systems are expected for fruit fresh attribute estimation?

1.6 Research Methodology:

This is a design-and-implementation based research. In essence, there are publicly available datasets where existing predictive models were built from by transfer learning on the models, then the full model was adapted into a software based system.

1: Literature Review and Dataset Selection

We have conducted a systematic literature review using terms for computer vision-related food quality estimation, transfer learning for application within smartphones, fruit post-harvest biology and ethylene interaction. From studies conducted we have selected three datasets to provide model training data: the Kaggle Fresh and Stale Fruits dataset, Fruits-360 Murean & Oltean, 2018 and another collection of fresh-quality labelled fruit images.

2: System Design

A three-tier architecture was then designed which comprises: Next.js frontend, FastAPI backend and a PostgreSQL database. A seven step scan pipeline was developed, which includes: image capture, fruit classification, freshness estimation, stored food values retrieving, remaining day prediction, ethylene storing conflict check and verdict display.

3: Implementation

In order to run the model over an actual smartphone application, we trained the MobileNetV2 using transfer learning and also designed and developed the actual web based application and also back-end and front-end, integrated with a number of modules for database handling (Postgres), user login and inventory tracking, and the ethylene interaction calculation mechanism for the system.

4: Testing and Evaluation

We then performed tests on individual parts of the implemented program and also on the complete application. Individual components and overall system were evaluated based on fruit classification correctness on unseen fruit and its freshness estimation compared with the actual available food science reference values and user friendliness on the system via smartphone application.

1.7 Scope of the study

Our scope is limited to fresh fruit - specifically 41 different types found in homes and Nigerian markets ranging from naturally occurring local varieties such as paw paw, native to Nigerian climate and sold locally as such to the imported varieties like apples and oranges, including the likes of pineapples, African star apples, grapes and many more.
We limited fruit for three main reasons;

1. Fruits spoil visually. Changes in color, texture and water presence is something which will be visible when taking photographs (Fu et al., 2022) which can not be done for meat, dairy or pre-cooked dishes as the food can appear perfectly unspoiled when actually there has been contamination.
2. The problem of ethylene interaction with fruit is only of concern for fruit and a function no consumer tool provides.
3. It is not practical within the time constraints of a university undergraduate project to fully test outside these strict boundaries.

The system only uses the camera on a cellphone. There is no sensor technology (temperature, gas or any form of IOT). Conditions such as room temp, refrigeration, and freezing, will be manually input via a selection tool with three options; room temp, refrigerator, and freezer.

1.8 Significance of the study

User Impact

Fresco is a useful, actionable tool consumers can use to minimize food spoilage in their own homes. The ethylene pairing tool solves a relatively uncommon but still a practical problem of storing different fruits together. One article clearly shows better information about freshness changes, usage patterns and waste practices (Principato et al., 2017).

Environmental and economic implications

Food waste leads to the loss of significant greenhouse gas emissions (FAO, 2019). Addressing waste reduction at a domestic level-the point at which most of it occurs in richer economies-requires intelligent information display methods and can thus offer potential to contribute towards reducing food waste.

Contribution to the academic field of research

The study presents a fully integrated system of transferring learning by using a well trained MobileNetV2 image classifier, the visual freshness score estimation and the ethylene adjusted storage life calculation to one final deployable application. It is the basis for creating future, more intelligent systems on platforms consumers can easily use.

1.9 Definition of Terms:

Term	Definition
Shelf Life	The period during which a fruit remains safe and acceptable to eat under specified storage conditions.
Dynamic Shelf Life	A shelf-life estimate that changes based on the fruit's current condition rather than a fixed date.
Freshness Score	A numerical value between 0.0 (spoiled) and 1.0 (fresh) representing the assessed visual condition of a fruit.
MobileNetV2	A lightweight CNN architecture designed for efficient image classification on mobile devices (Sandler et al., 2018).
Transfer Learning	A technique where a model pre-trained on a large dataset is adapted for a specific task by retraining only its final layers.
Ethylene	A natural plant hormone that accelerates ripening in fruits and causes some fruits to spoil faster when stored together.
Progressive Web App	A web application that can be installed on a smartphone and used offline, providing a native app experience without app store distribution.
Convolutional Neural Network	A deep learning model designed to process visual data by learning hierarchical features from images.

CHAPTER TWO: LITERATURE REVIEW

2.0 Introduction

2.1 Introduction

From a botanical definition, fruits are the mature ovary of seed-bearing plants, forming a particularly important part of the diet of humans thanks to their high concentration of vitamins, minerals, and dietary fiber. Nonetheless, fresh produce is highly perishable; after picking, the fruit continues its natural metabolic process, ending with senescence and ultimately spoilage. Food spoilage represents a challenge to the food system as it leads to the total loss of agricultural inputs, transportation energy and monetary value embodied in the fruit [1].

Therefore, assessing, accurately, and in a timely manner the level of freshness of fruits becomes crucial to food consumption control and the reduction of food wastage.

This link between the lack of accurate assessment of freshness and consumer-level food wastage is broadly documented, with global yearly loss/wastage of food up to 1.3 billion tons (FAO, 2019), estimated at approximately $940 billion (Gustavsson et al., 2017). The main cause of consumer food waste is the absence of reliable information to evaluate the actual state of the food acquired by households, such as that provided by static expiry date, assuming an idealized use that rarely occurs (Leena, 2024), and the total lack of labeling and packaging in developing countries like Nigeria for traditional fruit markets (Emegha et al., 2025), where the consumer is left with visual and sensory evaluation only (Nwafor, 2026). As a substitute for subjective judgment, information and technology have played an increasing role in bridging the gap and reducing waste by developing objectifying tools that assist and inform both consumers and producers.2.2. Fruit Spoilage: Causes and Consequences 2.2.1 Biological Causes of Spoilage The deterioration of fruit tissue is due to a mix of microbiological and physiological phenomena.

Fruit is prone to microbial attack, especially fungi, molds, and bacteria which start decomposing flesh through either mechanical defects or natural pathways, while enzymatic reactions drive the natural process of ripening, then leading to browning and senescence [4].

Postharvest fruits continue to respire, consuming reserved energy, water and oxygen to create carbon dioxide and so lose their structure and become overripe and ultimately spoiled [5]. From the perspective of a computer vision approach, it is important to notice that some of these natural processes, the physiological ones, have external visual indicators. Fungal spoilage: Fungi, molds and yeasts are responsible for 60-80% of fruit spoilage (Tournas, 2005).

It is commonly accepted that the development of external molds is 24–48 h after internal colonization begins (Tournas, 2005). Fresco considers only external signs of spoilage, such as color changes of bananas and apples, thus avoiding (though potentially missing) symptoms of interior rot.

Table 2.1: Visibility of Spoilage Types and CNN Detectability

Type of Spoilage	Visibility	Detected by CNN?
Surface mould	External	Yes
Colour changes	External	Yes
Texture changes (e.g., wrinkling)	External/Visible	Yes
Internal rot	Internal	No
Biochemical changes	Internal	No

2.2.2 Environmental factors influencing spoilage

Biological spoilage rates are largely a function of the environment. The most powerful influence is temperature. Even slight increases cause exponential increases in enzymatic reactions and the multiplication of many microorganisms. Indeed, an analysis of global sensitivity analysis for kinetic shelf-life models revealed that temperature variability accounted for the greatest portion of prediction error (Zhang et al. 2023).

Relative humidity promotes water loss and provides an environment to encourage fungal growth. Additionally, exposure to natural plant hormone, ethylene, speeds fruit ripeness and spoilage in susceptible species. Lastly, rough physical handling leads to bruising of the fruit and provides an opportunity for pathogens to invade (Ndraha et al., 2018).

Fresco takes into consideration a number of these conditions:

Table 2.2: Environmental Spoilage Factors in Fresco

Factor	Accounted for in Fresco?	How
Temperature	Yes	User selects storage method
Ethylene	Yes	Algorithmic compatibility checker
Humidity	No	Not implemented
Handling	No	Not implemented

2.2.3 Consequences of Poor Freshness Assessment

The socio-economic and environmental impact of ineffective freshness assessments can be severe. Households have commonly observed consumers throw away food that is still safe but out of an abundance of caution due to uncertainty surrounding its condition. Quested et al (2019) identified expiry date confusion as being a key cause for food waste deemed avoidable. Principato et al (2020) notes that expiry date confusion causes 32% of consumers to discard safe produce solely due to having a static label.

It is in situations with open market environments such as that in Nigeria that due to lack of labels, reliance upon Sensory heuristics becomes most apparent.

Emegha et al (2025) noted that insufficient food preservation knowledge and Poor Freshness Evaluation of Households contribute to food loss, which affects not only the nutritional status

(food variety) of households but also the socio-economic stability of the household (Nwafor, 2026). The cumulative environmental impact is immense as decomposing food in the Municipal solid waste results in substantial greenhouse gas emission (Ezeudoka et al., 2026). Fresco's (dynamic, condition based) approach attempts to avoid both a 'false urgency' (i.e. Wasting edible food) and a 'false confidence' (eating contaminated food).

2.3 Artificial Intelligence and Computer Vision

2.3.1 Machine Learning

Machine learning (ML) is considered to be a subtype of artificial intelligence that allows computers to learn how to identify and classify data, without being explicitly programmed for every individual task.

Machine learning falls under supervised learning which uses labeled input data that trains the algorithm to produce known output values (classification and regression) and unsupervised learning, which analyzes and finds patterns in unmarked data. As fruit deterioration is stochastic, complex and non-linear making it difficult to code a rule-set the application of machine learning towards food based assessment.

2.3.2 Deep Learning

Deep learning is a field of machine learning that deals with artificial neural networks that have multiple hidden layers (LeCun et al., 1998).

In general deep learning utilizes hierarchical approaches to feature learning. Unlike standard ML approaches where an engineer may be tasked with explicitly constructing certain features of the images (e.g. Specific colour Histograms, edge detections) deep learning algorithms extract features autonomously, identifying complex representations of the input data internally, and then building further more abstract features from them.

This ability of deep learning algorithms makes them especially powerful for analysing complex images such as those representing fruits.

This is in a large part due to deep learning algorithms directly managing the complexity of images, i.e. (high dimensionality of pixels), which are prevalent in computer vision tasks (Alarfaj et al., 2024).

2.3.3 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are a type of feed-forward neural network and are very effective at processing images, by learning spatial features hierarchically. The layers operate primarily through the application of convolutional operators: they feature learnable filters which produce a set of feature maps that act on the data across and through space (He et al., 2016). Non-linear (e.g., the Rectified Linear Unit - ReLU follows them) and pooling layers (usually average or maximum) to down-sample the image and the resulting feature maps.

Finally, the information in the maps is converted to a single vector that is then fed through fully connected (FC) layers to compute output probabilities.

The key to a CNN's success is its ability to learn hierarchies of features. The filters in the shallow convolutional layers can learn simple spatial features like edge orientations. The deeper filters build upon these features and can recognise patterns such as textures, then shapes, then objects (He et al., 2016).

For fruit quality assessment CNNs can be used for classifying a variety of fruit, identifying defective samples and assessing fruit ripeness and or maturity. An example relating to our field is research by Fu et al. (2022) whereby a number of CNNs are used to evaluate the citrus fruit based on their visual quality. In this work, the algorithm outputs a single score that represents ripeness on a 0-100% continuum, which implies that very early indications of spoilage days before they might be apparent visually could potentially be detected, thus representing the methodological framework utilized in the work.

2.3.4 CNN Architectures for Mobile Deployment Whilst

CNNs have achieved state of the art performance on many image classification tasks. These standard models (e.g., VGGNet [8], ResNet [13], Inception [14] etc.) can be excessively computationally intensive for deployment on edge devices like smartphones.

A research area that has focused on creating lightweight CNN architectures for mobile devices led to the development of a number of approaches that can reduce parameter size and computation load such as GoogLeNet [9], MobileNetV1 [10] and MobileNetV2 [11]. MobileNetV2 can efficiently represent information by employing depthwise separable convolutions, which split a standard convolution into a spatial depthwise convolution and a

channel wise pointwise convolution operation to reduce computation costs considerably (Howard et al., 2017). These network blocks also utilize inverted residual structures and linear bottom necks which effectively prevent the loss of information in the channels.

Table 2.3: Comparison of CNN Architectures for Mobile Deployment

Architecture	Key Feature	Strength	Limitation
MobileNet (2017)	Depthwise separable convs	Lightweight	Lower accuracy than deep nets
MobileNetV2 (2018)	Inverted residuals	Better efficiency/accuracy balance	Modest accuracy trade-off vs ResNet
EfficientNet (2019)	Compound scaling	High accuracy	More complex architecture
EfficientNet-Lite	Optimised for mobile	Good mobile performance	Newer, less deployment history

Fresco utilises MobileNetV2, due to its efficient trade-off between computational cost and classification accuracy, making it ideal for a Progressive Web App for mobile devices and enabling fast consumer hardware inference without degrading user experience or relying upon expensive cloud-compute offloading (Hu et al. 2024).

2.3.5 Computer Vision

The computer vision task of analyzing and classifying visual inputs as a whole or for multiple objects within. Tasks may vary by the level of specificity required from pixel up to the object label assigned. Tasks can include image classification (assigning a single label/ category), object detection (placing bounded boxes around all objects and assigning them a label), and segmentation (classifying each pixel of an image with foreground/background labels).
Fresco only performs image classification of a single fruit image. The system expects the user to place a single piece of fruit centred on the camera. All input is then processed to provide a single

output indicating the type of fruit and the stage of aging (e.g., 5-10 day Banana). The system does not involve object detection or segmentation and does not support images with overlapping fruit.

2.3.6 Transfer Learning

Training a CNN entirely from scratch necessitates large amounts of data, and thus cannot be expected as input to our Progressive Web App. Transfer learning avoids the necessity to train large networks by initializing network weights using pre-trained weights of a CNN trained upon a large dataset (e.g. ImageNet, a dataset of 1.28 million images across 1000 categories) to assist in training of network for a narrower, specific task (Tajbakhsh et al. 2016). The initial layers of the network are utilized for its general visual feature learning (lines, curves, textures, color gradients), and it requires far less task specific training for the later layers.
Frescoe utilizes transfer learning by loading the ImageNet-pre-trained MobileNetV2 base layers and freezing them to preserve the visual feature extraction performed. A fully-connected layer is appended to output classifications to 14 classes corresponding to stages of aging and categories for specific fruits, Apple, Banana, Carrot, Tomato, and Expired (using dataset images with labels taken from Fruit-360 \[Murean and Oltean 2018]). Next, the last of the base convolution layers were un-froze at a very low learning rate, to refine feature representations with respect to the agricultural morphology required.

2.4 Background to existing food-freshness classification methods

2.4.1 Sensory-Based Inspection

In the Nigerian open-market environment that lacks printed expiration dates on fresh produce the most relied on measure is through sensory inspection. This involves using the five human senses to determine the condition of the fruit through: sight (visual discoloration, mold and decay), smell (off odor's), and touch (unnatural softness). Although these are easily accessible, universally available and free without special equipment required, sensory inspection relies heavily on the individual, with highly subjective and disparate classifications.

Most importantly, the senses react to decay-based conditions rather than proactively identify such conditions.

The study by Tournas(2005) clearly demonstrated that by the time mold appears on a fruit, there already was a fungal growth taking place inside the fruit that predates visual inspection for at least 24-48 hrs. CNN-based classification provides a reliable baseline that detects decay in a more forward-leaning manner.

2.4.2 Sensor-Driven/IoT-Based monitoring

The cutting-edge technology today for dynamic, forward-looking shelf-life prediction is based on physical sensing elements and the Internet of Things (IoT). Various types of sensors continuously monitor the internal environment through, for example, thermal, humidity and specific gas-analysis sensors. Wang et al. (2025) created a sensing array combining multiple physical sensors (specifically e-nose gas sensors and continual monitoring thermal sensor data) that analyzed various types of fruit to predict food perishability.

Table 2.4: Characteristics of Multi-Sensor IoT Fusion Monitoring

Aspect	**Detail**
Method	Multi-sensor fusion
Accuracy	94.5%
Hardware required	Gas sensors, temperature monitors, IoT nodes
Consumer accessibility	Low (requires external hardware infrastructure)

Other methods (Sahu et al. 2020) propose the use of edge computing IoT devices on microcomputers such as Raspberry Pi to monitor food quality offline. While it has impressive accuracy, hardware prerequisites in their system hinder its widespread adoption amongst ordinary consumers. Hence Fresco does not incorporate any physical hardware into the devices to be more accessible to the average consumer using readily available smartphones rather than achieving optimal accuracy.

2.4.3 Hyperspectral Imaging

Hyperspectral imaging collects light in many hundreds of spectral bands from a sample over a wide range of spectra including and beyond the visible light. This allows camera systems to observe biochemical changes such as the decline of chlorophyll, production of sugars and the depletion of moisture. Patel et al. (2024) also proved that hyperspectral image analysis of fresh produce offers prediction accuracies of greater than 95%.

Table 2.5: Characteristics of Hyperspectral Spoilage Imaging

Aspect	Detail
Accuracy	95%+
Hardware barrier	Requires specialised, high-cost research-grade cameras
Consumer accessibility	Extremely low

While hyperspectral imaging is exceptionally powerful because it can detect internal rot and bruising that RGB cameras cannot see, the equipment is entirely unrealistic for consumer deployment. Fresco relies on standard RGB smartphone images, accepting the limitation of only analysing visible surface degradation.

2.4.4 Computer Vision Approaches

Several studies have deployed computer vision systems for visual quality grading. Fu et al. (2022) used YOLO and ResNet architectures to grade citrus fruit freshness, mapping visual data to a continuous 0–100% freshness scale rather than categorising fruit strictly as "fresh" or "spoiled." They found that CNNs could reliably detect early-stage degradation prior to human inspectors, though the models were sensitive to low-lighting conditions. This continuous scoring methodology serves as the conceptual basis for Fresco's fuzzy-logic-inspired 0.0-to-1.0 freshness assessment (Vasquez et al., 2024).

Yamamoto et al. (2020) focused on surface defect detection in citrus fruits using lightweight CNN architectures, establishing that highly complex networks were not strictly necessary for agricultural defect detection. Building on this, Hu et al. (2024) optimized an EfficientNet architecture for mobile vegetable quality grading, validating that modern edge devices possess sufficient computational capacity to run real-time AI inference. Subari et al. (2024) similarly demonstrated the successful deployment of MobileNetV2 for fruit freshness detection on Android devices. These studies confirm the fundamental feasibility of Fresco's deployment strategy: performing convolutional freshness assessments directly within consumer smartphones. Furthermore, the dataset foundation for many of these image-based models, including Fresco, relies heavily on standardised image sets such as Fruits-360 (Mureșan & Oltean, 2018).

2.5 Post-Harvest Biology: Ethylene Interactions

2.5.1 Ethylene Production

Ethylene (C_2H_4) is a naturally occurring gaseous plant hormone that plays a central role in regulating fruit ripening and senescence. Fruits are physiologically classified as either climacteric (experiencing a spike in respiration and ethylene production during ripening) or non-climacteric. The production of ethylene triggers biochemical pathways that soften tissue, convert starches to sugars, and alter peel colour. As documented by Watkins (2006), certain fruits produce substantial quantities of this gas as they age. High ethylene-producing fruits include Apples, Avocados, Bananas, Mangoes, Papayas, Pears, and Tomatoes. Fresco encodes this biological data into its database to proactively identify potential storage hazards.

2.5.2 Ethylene Sensitivity

Many fruits are highly sensitive to exogenous ethylene. When exposed to the gas emitted by neighbouring producers, these sensitive fruits experience accelerated ripening, rapid quality deterioration, softening, and accelerated fungal susceptibility. Fruits classified as highly sensitive to ethylene include Apples, Bananas, Grapes, Kiwis, Mangoes, Oranges, Strawberries, and Watermelons. Fresco leverages this biological data to actively warn users against storing these items in close proximity to producers.

2.5.3 Storage Implications

At the commercial scale, the post-harvest supply chain meticulously manages ethylene interactions. Commercial cold storage facilities actively separate incompatible commodities and precisely control atmospheric conditions to prevent accelerated spoilage. However, these practices rarely translate to the household level. Consumers, generally unaware of these invisible chemical interactions, routinely store incompatible items together in mixed fruit bowls or crisper drawers, inadvertently accelerating their own food waste. By implementing an algorithmic compatibility checker, Fresco seeks to translate this industrial knowledge into a practical consumer tool.

2.6 Summary of Related Works

The following table synthesises the critical academic literature informing the foundational methodologies, technological constraints, and biological principles underlying the Fresco system.

Table 2.6: Summary of Critical Academic Literature and Related Works

Author(s)	Year	Focus	Method	Key Result	Constraint	Relevance
Fu et al.	2022	CNN grading	YOLO/ResNet	Detected early degradation	Light sensitivity	Freshness score basis
Sandler et al.	2018	Mobile Arch	MobileNetV2	Low compute cost	Accuracy trade-off	Core model framework
Watkins	2006	Biology	Ethylene analysis	Quantified production	Biological study	Compatibility foundation
Vasquez et al.	2024	Prediction models	Fuzzy vs Arrhenius	Fuzzy logic handles noisy sensor data better	Requires manual rule mapping	Justifies fuzzy-inspired approach

Table 2.6: Summary of Critical Academic Literature and Related Works

Author(s)	Year	Focus	Method	Key Result	Constraint	Relevance
Fu et al.	2022	CNN grading	YOLO/ResNet	Detected early degradation	Light sensitivity	Freshness score basis
Sandler et al.	2018	Mobile Arch	MobileNetV2	Low compute cost	Accuracy trade-off	Core model framework
Watkins	2006	Biology	Ethylene analysis	Quantified production	Biological study	Compatibility foundation
Vasquez et al.	2024	Prediction models	Fuzzy vs Arrhenius	Fuzzy logic handles noisy sensor data better	Requires manual rule mapping	Justifies fuzzy-inspired approach
Leena	2024	Expiration Labelling	Systematic review	Static date fall in in stochastic supply chains	Diagnostic, no tech solution proposed	Core problem diagnosis
Watkins	2006	Post-harvest biology	Ethylene analysis	Quantified fruit ethylene production & sensitivity	Biological study	Foundation of compatibility engine
Wang et al.	2025	IoT monitoring	Multi- sensor fusion	94.5% prediction accuracy	Expensive physical hardware required	Highlights hardware accessible gap
Tournas	2005	Spoilage detection	Mycological study	Visible mould signifies existing internal rot	Biological study	Proves sensory inspecti is relative

Author(s)	Year	Focus	Method	Key Result	Constraint	Relevance
Hu et al.	2024	Mobile deployment	EfficientNet edge AI	Validated real-time smartphone interference	Hardware specific optimization	Validates smartphone ec inference
Zhang et al.	2023	Shelf-life modeling	Sensitivity analysis	Temperature variation drives prediction errors	Math-heavy simulation	Highlights need for storage context
Wibowo et al.	2022	Shelf-life reasoning	Fuzzy inference	Natural language rules increase user trust	Static rule sets	Validates UI interpretability
Kumari	2025	ML prediction	Random Forest	96% accuracy on historical data	“Black box” impacts user trust	Justifies transparent decision engine
Subari et al.	2024	Fruit freshness	MobileNetv2/ Andriod	99.6% test accuracy on selected subsets	Limited to specific fruit variants	Validates exacts technol decision engine
Sahu et al.	2020	Edge computing	IoT on Raspberry Pi	Enabled offline food monitoring	Required physical computing node	Validates offline capacibilities
Emegha et al.	2025	Food waste in Nigeria	Socio-cultural survey	Highlighted poor preservation knowledge	Regional focus	Contextualises the Nige problem space

2.7 Research Gaps

Synthesising the literature surveyed there appear to be distinct gaps in the literature in respect to the consumer face-integration of already existent technologies involved in assessing fruit freshness.

Integration: Existing computer vision algorithms have traditionally seen identification and quality assessment as separate subtasks. In none of the studies surveyed had a system automatically identified the fruit in a photograph, continuously scored its appearance for freshness, predicted a dynamic estimate of its remaining shelf-life or accounted for its ethylene interaction.

Accessibility: Most work in the surveyed literature has been undertaken to achieve highest detection accuracies using highly specialised hardware. Such devices require either a multi-sensor IoT integration (Wang et al., 2025) or are hyperspectral camera-based (Patel et al., 2024), two devices which are largely inaccessible to a typical household.

Ethylene: In the commercial logistics sector of fruit supply chains, due awareness is maintained as ethylene production and absorption is generally considered, yet the integration of this consideration to the consumer-facing AI systems concerned with fruit freshness seems less prominent in the surveyed literature.

2.8 Chapter Summary:

The implications of the surveyed literature for Fresco.

From the literature surveyed, traditional systems for fruit freshness detection are unlikely to lead to reductions in household food waste. With static use-by dates unable to cope with the "stochasticity in thermal histories inherent in the food supply chain" (Leena, 2024), and reliance on subjective and reactive visual inspection being fallible; advanced IoT and hyperspectral detection systems represent a dynamic yet prohibitive solution, due to costly hardware, rendering their domestic application currently unlikely. In contrast the software systems needed to facilitate computer vision through high-performance systems lack hardware dependency: MobileNetV2 has demonstrated its capacity as an edge-optimised model, continuous CNN freshness scoring has been used to dynamically track loss of appearance, and well-established knowledge states that it's possible to substitute a discrete, rather than discrete kinematic evaluation with an uncertainty-tolerance categorical approach (Vasquez et al., 2024). In summary the key gap is integration and accessibility. Fresco aims to breach this gap by combining MobileNetV2 transfer learning with a fuzzy logic inspired approach to model freshness estimation. A curation of post-harvest data, combined with a dynamic interpretation approach considering the fruits interaction with ethylene within the supply chain, and ultimately serving the user via a mobile Progressive Web Application provides this unique integrated and accessibility solution to reduce unnecessary food waste.

CHAPTER THREE: METHODOLOGY

3.1 Introduction

The purpose of this section is to describe the adopted methodological strategy in designing, implementing and deploying the Fresco system. Fresco was created following a design-and-implementation strategy - involves constructing an information technology (IT) system and then critically assessing its performance. The development process was divided into four major stages, firstly a review of literature, second the system architecture design and communication flow of major components and third implementation including, the data capture, data preparation, the system and finally testing of the produced system and evaluating it against pre-defined success metrics.

The methodology incorporates a transfer learning approach in training a pre-trained CNN model to detect variety and variety maturation stage.

Use of fuzzy logic in evaluation of the model results and generation of a progressive score. These methods can be broken down into 6 modules: system architecture, data acquisition, data preparation, modeling, score calculation, deployment. Chapter starts with the overarching system and how 7 step scan pipeline works followed by the sources of data, steps in data preparation, models developed for fruit identity and a freshness classification and remaining shelf life estimation. Lastly explains how a score is calculated and how the system is deployed in the market.

3.2 SYSTEM ARCHITECTURE

The system architecture consists of the major interacting components within the Fresco application and how they interact to process and interpret an input fruit image and display freshness-related information.

3.2.1 Architecture

A three-tiered architecture was implemented in the Fresco system to facilitate abstraction between the physical client device (view), processing component (logic), and storage (data). This has separated its key components into, 1. Presentation tier, 2. Application tier and 3.Data tier.

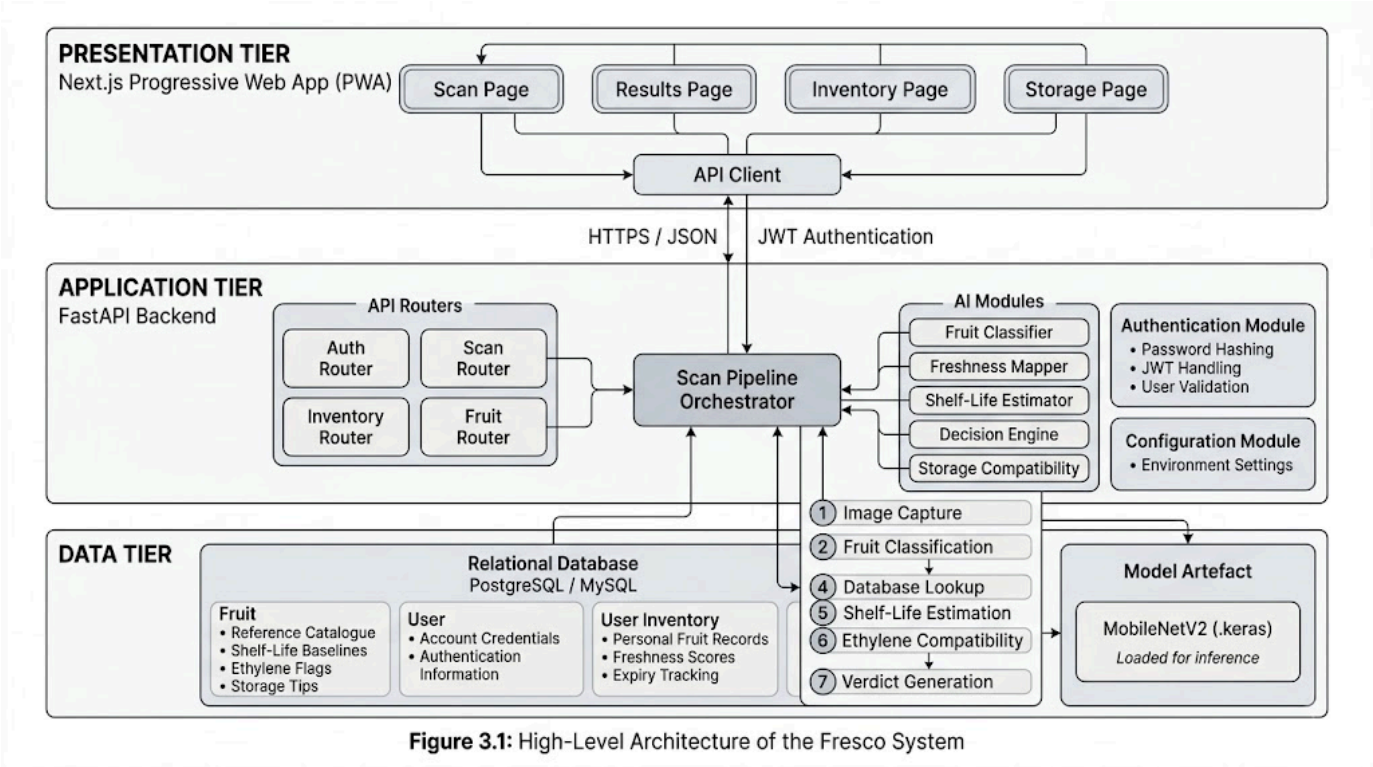


Figure 3.1: High-Level Architecture of the Fresco System

The presentation tier is a Next.js Progressive Web Application that handles the user interaction with the system. This tier includes aspects such as image capturing (either using the device camera or file upload), presentation of the results of the scan, personal inventory management and the check of storage compatibility. The frontend communicates with the application tier via HTTPs requests.

The application tier is a FastAPI backend server that receives the requests from the frontend and manages the image scanning process, the image uploading and the classification by the AI model, along with the related processing steps, and provides a structure response. It hosts API routers, the AI pipeline orchestrator, AI modules, the authentication module and the configuration.

The data tier includes a relational database and a model artefact. The database stores reference data of fruits, user accounts, personal inventories and scheduled notifications. The model artifact refers to the trained MobileNetV2 classifier, serialized into a Keras file that the backend loads into memory upon application startup and it is loaded into memory for the entire lifetime of the server process.

3.2.2 Systems

Model Artefact: the trained MobileNetV2 model was saved as a serialized Keras file. When the application starts, this model is loaded into memory by the backend server where it gets warmed-up. The model artifact can be treated as a static file and is not re-trained at runtime by

the application. Once a scanning request is received from the frontend, the model is used to perform inference with respect to the preprocessed image.

3.2.3 Data Flow and Processing Pipeline

A 7-step scanning pipeline is executed from the moment an image upload is made to the response of the scanning request. The pipeline processes the user input and returns all necessary information.

1. Image Capture and Upload: The user captures or uploads an image to be processed. The frontend converts the input image to JPEG format and sends it to the scan endpoint of the backend via multipart-form-data format. The image upload is validated at the backend checking the MIME type and the size of the file, then saved with a randomly generated file name.

2. Fruit Classification: The image taken is loaded and then preprocessed to be passed to the classifier. In this step the image is resized to 224 224 pixels in resolution and the color space is adjusted to RGB. Image normalization from [0, 255] range to [-1, 1] range is done to match the requirements of MobileNetV2. Then, the loaded model performs inference on the preprocessed image and a list of probabilities regarding each of the 14 class categories is produced. The maximum probability value, associated to a specific class, indicates the fruit detected and its confidence in the detection.

3. Freshness Scoring: The classified fruit-class label is translated into a value between 0.0 to 1.0 on the freshness scale using a reference table containing calibrated score values for all the classes and defined after consultation and annotation of real data. The freshness score is then mapped into the categorical values Fresh, Slightly Aged, Ageing, Old or Spoiled.

4. Database Lookup: The fruit identified is searched in the fruit references table by use of two methods in sequence: first exact case-insensitive match and then substring matching if exact match does not work. If a reference object including the lifespan information and storage preferences is found, it's loaded and passed to the next step.

5. Shelf-Life Estimation: a shelf-life estimator calculates the remaining days of shelf life using three storage methods (room temperature, refrigeration and freezing). The calculation takes in consideration the basic lifespan of the fruit, the calculated freshness score, a storage factor based on the storage method and fruit storage optimum temperature and finally a ripeness score also

based on the freshness. Three values representing the remaining shelf-life in days are returned and passed to the next step.

6. Ethylene Compatibility: The detection engine then analyzes if the detected fruit produces/accepts ethylene production and then,if the system is currently carrying user inventory, checks if the identified fruit can coexist with elements already in its inventory. In case an anomaly is found, a notification will appear explaining the expected consequence.

7. Verdict Generation: The decision engine then combines the calculated freshness score and normalised days remaining to produce a final status of FRESH, EATSOON, EATTODAY or SPOILED. From the resulting status a suggestion message is displayed to the user in form of a prepared string and the best storage method suggested is derived from the maximum estimated shelf-life, provided along the remaining information. A structured response is created to be sent back to the frontend.

3.3 Data Acquisition

The below section describes the datasets that were used for training the fruit identification and freshness detection models.

3.3.1 Data Sources

The second part of this project involved acquiring suitable training data sources for both the fruit identification and freshness detection models.

Data Source 1: Fruits-360. The Fruits-360 data set consists of 90,483 fruit images (100x100 resolution) categorized into 131 fruit-like classes, provided by Murean and Oltean (2018), and downloadable from an open repository on GitHub. This data set was used as the primary training data set for the fruit identification portion of the Fresco system. It offers a variety of fruit-classes with great images for each, and it served as the base for modifying it into 14 age-classes that are needed to create a system that performs fresh-ness classification.

Data Source 2: Fresh and Stale fruits and vegetables. A data set that was downloadable from Kaggle consisting of around 3,000 images of fresh and stale fruits and vegetables with various resolutions and formats (JPEG and PNG), as well as 4 fruit-type classes that have been used for training in this project: Banana, Carrot, Tomato and Apple. The Fruit data set is an ideal complement for the Fruits-360 data set as it labels the photos into "Fresh" and "Stale"categories,

this is precisely what is desired to use to implement the Fresco systems age-classification phase. Additional Curated Images: no external, homemade images were used beyond the provided ones. Fruits-360 and the Kaggle Fresh and Stale Fruits and Vegetables data sets comprise sufficient data for training the project models during development phase of Fresco.

3.3.2 Data Description

The obtained data from each source was reorganized into 14 different class values which represent the product itself combined with a determined age state for Fresco system functionality.
Classification Mapping: In order to categorize the models classes, both data sets were mapped into the Table 3.1 below

Table 3.1: Definition of Age and Freshness Target Classes

Category	Age/Freshness Class Definition
Apple	Apple (1–5), Apple (5–10), Apple (10–15)
Expired	Visibly spoiled specimens

The age ranges, such as (1-5) or (5-10), represent the approximate age of the fruit in days. These ranges were derived from the freshness annotations available in the datasets. The "Expired" class contains images of fruits that are visibly spoiled, regardless of the fruit type.

Image Characteristics

The images in the combined dataset had the following characteristics:

Table 3.2: Technical Specifications of Source Images

Image Property	Technical Specification
Colour Space	RGB (Standardised)
Input Pipeline Target	Standardised 224 × 224 pixels

LABELLING:

The original dataset contained fruit classification labels whereas the Kaggle dataset contained freshness based classification labels. To create the 14 class target categories the freshness based classifications from the Kaggle dataset were interpreted as an age group and assigned appropriately. Fruits for which the Kaggle classification was unavailable were interpreted with an eye towards existing knowledge on fruit maturity and also via visual inspection.

The targeted categories will thus encode both identity and the desired characteristic (i.e., target classes will have names like Banana(5-10)) enabling the classifier to produce two outputs.

CLASS IMBALANCE:

Some classes had far more images than others in the dataset and likewise the Kaggle dataset had a relatively small number of images for each of its four classifications. The class distribution was analysed whilst preparing the datasets and the training/ validation and test sets were balanced according to this relative distribution.

3.4 Data Preparation

Here are the steps applied to the images so that the resulting dataset could be fed into an image classifier - Raw images sourced from both datasets were cleaned and put into a standard format and validated, then were split in a ratio for training and validation and then subjected to data augmentation.

3.4.1 DATA CLEANING, LABELLING AND ORGANISATION

Raw images were sourced from two publicly available image datasets.

Data from the datasets was combined for classification and organized as a unified dataset for classifier use.

REMOVAL OF INVALID DATA .images were tested for duplicates and/or corrupted images -Images that could not be opened/ processed were deleted.

CLASS MAPPING.

The original labels for images from both datasets were reformed to create 14 target classes from the different classifications.

All target classifications are illustrated in Table 3.2.

Table 3.3: Fruit Categories and Associated Age-Range Classes

Fruit Category	Age-Range Classes
Banana	Banana (1–5), (5–10), (10–15), (15–20)
Other	Expired

The relevant fruit categories (Apple, Banana, Carrot, and Tomato) were extracted from both datasets. Age-range labels were assigned based on the available freshness information. The Fruits-360 dataset provided age-stage labels directly, while the freshness labels from the Kaggle dataset were mapped to the project's age ranges.

An "Expired" class was created from visibly degraded specimens across the selected fruit categories. All images were then grouped into the 14 target classes to create a unified dataset with consistent labels.

Image Standardisation. The images were standardized to a consistent format. All images were converted to the RGB colour space to ensure consistent colour-channel representation. The images were resized to 224 × 224 pixels to provide a consistent input size for the MobileNetV2 classifier.

3.4.2 Data Validation

Several validation checks were performed to ensure the prepared dataset was suitable for model development.

Table 3.4: Data Validation and Quality Control Checks

Validation Check	Objective
Label Consistency	Verify target class assignment

Each image was checked to ensure it was a valid file that could be opened and processed. The colour space was verified and converted to RGB where necessary. All images were resized to

224 × 224 pixels. The class labels were reviewed to confirm that each image was assigned to the correct target class.

3.4.3 Data Splitting

The unified dataset was divided into three subsets for model development and evaluation.

Table 3.5: Allocation of Dataset Splits

Split	Proportion	Purpose
Test	10%	Final performance metrics

The 80/10/10 split was applied to the dataset. The training set was used to update the model weights during training. The validation set was used to monitor performance and select hyperparameters. The test set was reserved for final evaluation after training was complete.

Stratification was applied during splitting to maintain the class distribution across all three subsets. This ensured that each class was represented proportionally in the training, validation, and test sets.

A separate held-out test set of 280 images (20 images per class) was also reserved for final evaluation. This set was kept completely separate from training and validation and was not used during model development.

Note on Dataset Sizes. The combined dataset contained approximately 4,000 images after cleaning and organisation. This resulted in approximately 3,200 images for training, 400 for validation, and 400 for testing. These numbers are approximate and reflect available data from both source datasets.

3.4.4 Data Augmentation

Data augmentation was applied during training to introduce realistic variation into the training images. This helps the model generalise to images captured under different conditions.

Table 3.6: Applied Data Augmentation Techniques

Technique	Parameter	Purpose
Zoom	±10%	Scale robustness

These augmentations were chosen to reflect the types of variation expected in real-world use. Users may hold their phone at different angles, capture images under different lighting, and frame fruits at different distances. The augmentations were applied randomly during training to increase the effective size of the dataset.

Augmentation was applied only to the training data. The validation and test sets were not augmented, as they are intended to evaluate performance on unmodified images.

3.5 Models

This section describes the three models that form the core of the Fresco system. The models operate sequentially to transform an input fruit image into a freshness score, shelf-life estimate, and final recommendation.

3.5.1 Model 1: MobileNetV2 Convolutional Neural Network

Purpose

Model 1 is the image classification component of the system. It identifies the fruit type and its age or freshness stage from the input photograph.

Input

The model receives an RGB image resized to 224 × 224 pixels. The pixel values are normalised to the range [-1, 1] to match the preprocessing used during the model's pre-training on ImageNet.

Architecture

The model uses the MobileNetV2 architecture as its base. MobileNetV2 was designed for efficient image classification on mobile and embedded devices (Sandler et al., 2018). It uses depthwise separable convolutions and inverted residual blocks to reduce computational cost while maintaining accuracy.

The base model was pre-trained on ImageNet, which contains 1.28 million images across 1,000 classes. A custom classification head was added on top of the base network to produce predictions for the 14 target classes used in this project.

Transfer Learning Process

The model was trained using transfer learning in two phases.

Phase 1: Frozen Base. The MobileNetV2 base layers were frozen, meaning their weights were not updated during training. Only the newly added classification head was trained. This phase ran for 10 epochs with a learning rate of 1×10^{-3}.

Phase 2: Fine-Tuning. The top 30 layers of the base network were unfrozen and trained at a lower learning rate of 1×10^{-5} for 15 epochs. This allowed the pre-trained features to be slightly adjusted to the fruit domain without causing catastrophic forgetting.

Table 3.7: Transfer Learning Training Phases

Phase	Epochs	Learning Rate	Layers Trained
Phase 2: Fine-Tuning	15	1×10^{-5}	Top 30 layers

Training Parameters

The model was trained using the Adam optimiser with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The loss function was categorical cross-entropy, which is appropriate for multi-class classification problems.

The categorical cross-entropy loss is defined as:

$$L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

where N is the number of classes, y_i is the true label indicator, and $\hat{y}_i$ is the predicted probability for class i.

A batch size of 32 was used during training. The batch size was reduced from an initial 64 due to GPU memory constraints during the fine-tuning phase. Training was performed on Google Colab using an NVIDIA T4 GPU with 15 GB VRAM.

The final layer uses a softmax activation function, which produces a probability distribution over the 14 target classes.

Output

Model 1 produces a predicted class label and a confidence score representing the probability of that prediction. For example, the model might output "Banana (5-10)" with a confidence of 0.91. The predicted class is passed to Model 2 for freshness scoring.

3.5.2 Model 2: Fuzzy-Logic-Inspired Freshness Mapper

Purpose

Model 2 converts the discrete age-stage prediction from Model 1 into a continuous freshness score between 0.0 and 1.0. The score represents the visual condition of the fruit, where 1.0 indicates perfectly fresh and 0.0 indicates completely spoiled.

Input

The primary input is the class label from Model 1, such as "Banana (5-10)".

Mapping Strategy

The class label is mapped to a continuous freshness score using a calibrated lookup table. The mapping was derived from the freshness annotations available in the training data and validated against human judgement.

Table 3.8: Calibration of Class to Freshness Score Ranges

Age-Range Class	Score Range	Category
Expired	0.00 – 0.19	Spoiled

The mapping can be expressed as:

$$S_{\text{freshness}} = \text{score\_map}(\text{class\_label})$$

where S_freshness is the resulting freshness score and score_map is the lookup structure that assigns a score value to each class label.

Example Mappings

Table 3.9: Model Mapping Logic Examples

Predicted Class	Freshness Score
Expired	0.10

Output

Model 2 produces a continuous freshness score between 0.0 and 1.0. This score is passed to Model 3 for shelf-life estimation.

3.5.3 Model

3: Multi-Factor Shelf-Life Estimator Purpose Model 3 estimates remaining shelf life in days, with calculations conducted independently for each storage condition: room temperature, refrigerated, and frozen. Inputs There are four model 3 inputs: FruitID (obtained from Model 1), FreshnessScore (obtained from Model 2), the user-specified StorageMethod, and FruitReferenceData from the database. The fruit reference data contains baseline shelf life values for each storage method, the fruit's "ideal" temperature range, etc.

Main equation Edays = BS1.5 Tf Rf E_days is estimated remaining shelf life (days).

B is base shelf life (fruit + storage method) of the form S is freshness score from Model 2. Tf is the temperature factor. Rf is the ripeness factor. Temperature factor Tf will penalize storage conditions outside the "ideal" temperature range.

Optimal storage temperatures are hard-coded to 22 C (room temperature), 4 C (refrigerated) and -18 C (frozen).

Tf will be 1.0 if temperature is in the ideal range: Tf=1.0 when in range Tf = 1.0 If temperature is outside the "ideal range": Tf = max(1.0-0.02d, 0.5) (d is amount out of range in deg C) (Note: minimum is 0.5. This prevents shelf life from being reduced > 2X) Tf=max(1.0-0.02*d, 0.5) For the freezer: Tf = 0.95 (reflects slight quality loss for freezing and thaw) Ripeness factor Reflects ripening.

Table 3.10: Ripeness Multipliers for Shelf-Life Estimation

Freshness Score	Ripeness Stage	Multiplier
< 0.50	Overripe	0.30

Worked Example

To illustrate the calculation, consider a banana with a freshness score of 0.72 stored in the refrigerator.

The fruit reference data for bananas gives a refrigerator-based shelf life of 7 days and an optimal temperature range of 13°C to 15°C.

The freshness factor is:

$$S_{1.5} = 0.72_{1.5} = 0.611$$

The temperature factor for refrigerator storage (4°C) is 0.82, as the temperature is below the optimal range.

The freshness score of 0.72 falls in the Nearly Ripe category, giving a ripeness factor of 0.85.

The estimated shelf life is:

$$E_{\text{days}} = 7 \times 0.611 \times 0.82 \times 0.85 = 2.98 \text{ days}$$

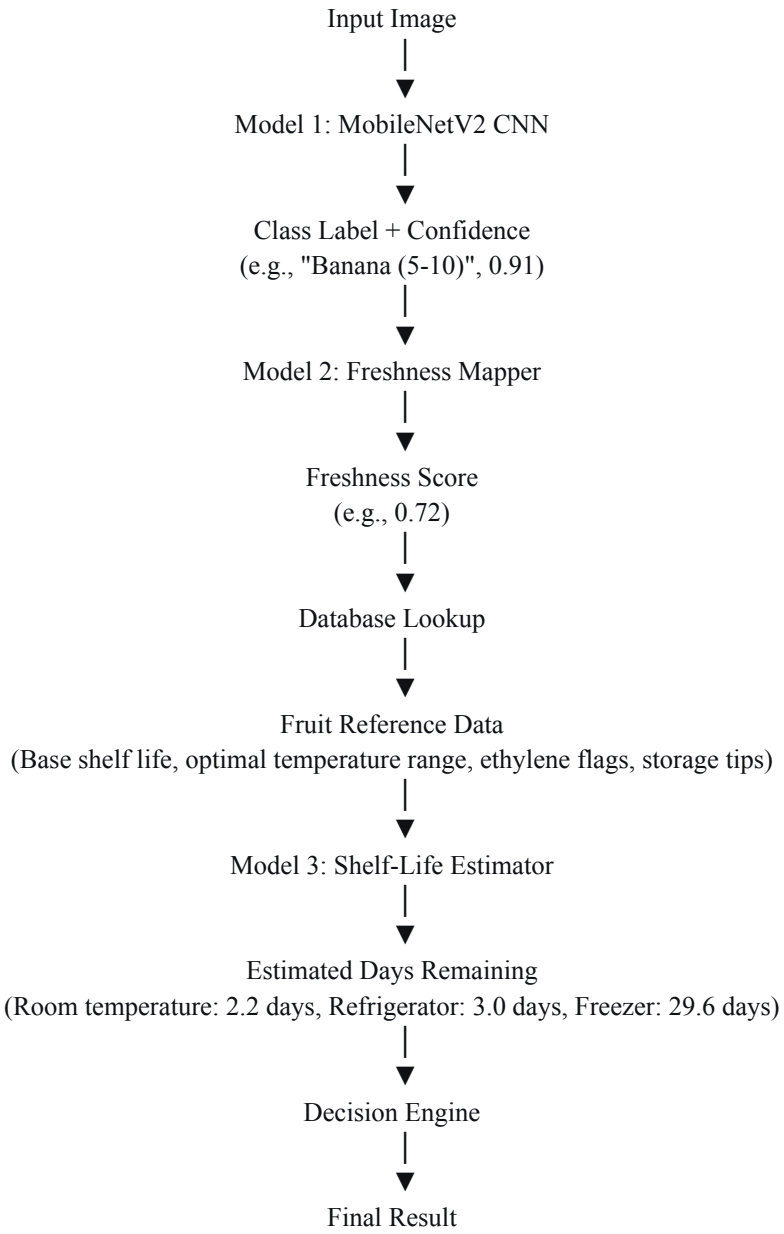
Output

Model 3 produces estimated days remaining for each of the three storage methods. These estimates are passed to the decision engine for final verdict generation.

3.5.4 Model Interaction and Processing Pipeline

The three models operate as an integrated pipeline. Each model receives input from the previous stage and passes its output to the next stage.

Integration of the Fresco Modelling Pipeline



(Status, days remaining, recommended storage, recommendation string, ethylene advisory)

The pipeline flows from the image submission: Model 1 detects and classifies the fruit image with its class label and confidence value; Model 2 maps the class label to a continuous freshness score; a database lookup determines reference data for the fruit; Model 3 uses the freshness score and reference data to predict remaining shelf life for all storage methods; finally, the decision engine combines these values for its verdict. The decision engine determines the stored verdict using a composite score as:

C=(S0.6)+(D norm 0.4)

where D_norm is the ratio of estimated days remaining by base shelf life for any method.

A highest C value yields a recommendation. Final stored verdict will consist of verdict, remaining days for best method, storage method, recommendation string and (optionally) ethylene data.

3.5.5 Ethylene Compatibility Engine

The algorithm requires two disjoint fruit sets: fruits that produce ethylene (19), referred to hereafter as ETHYLENEPRODUCERS, and fruits that deteriorate measurably faster when exposed to ethylene (24 fruits, referred to hereafter as ETHYLENESENSITIVE). An undirected graph is constructed by testing all pairs from each set, a 456 comparison process. Bidirectional edges are formed between producer fruits P and consumer fruits S.

A total of 456 - 5 = 451 edges are formed for P ∈ETHYLENEPRODUCERS and S ∈ETHYLENESENSITIVE.

An edge between fruits i and j means that storing fruits i and j together will deteriorate the storage life of at least one of them, or more specifically, i and j is one such fruit pair as above. Conflict graph as described here is represented as adjacency list, a dictionary mapping fruits'

name to the set of conflicted fruits. The pairwise check takes a short O(1) duration given an adjacency list.

3.6 Determination of Freshness Score (0–1)

The algorithm first estimates age and state based on fruits' images, and then converts this estimate into numerical values that are used to find suitable storage conditions. This section details how this estimation and value conversion process works, how it works.

3.6.1 Freshness Score

As mentioned above, the raw outputs of Model 1 and Model 2 are fruit's age category and freshness score, S, (0-1).

Age and state information is generated from the output of a first, single trained Model that acts upon the training images.

The age class and state, such as "Banana (5-10)" is then fed into a second "fuzzified" Model 2 that, based on training images, produces the S for the fruits, where a score close to 1.0 represents a fresh fruit and a score close to 0 represents the other extreme. The conversion to S, is formulated as S = score_map(class)

Example Mappings:

Table 3.11: Examples of Numerical Score Calibration

Predicted Class	Freshness Score
Expired	0.10

These values illustrate how different predicted classes map to different freshness scores.

3.6.2 Freshness Category Mapping

The continuous 0–1 score is grouped into five categories to make the result easier for users to understand. Each category represents a general freshness condition and provides guidance on how the fruit should be handled.

Table 3.12: Interpretation of Freshness Categories

Score Range	Category	Interpretation
0.00 – 0.19	Spoiled	No meaningful shelf life

The category is derived from the numerical score. The score is calculated first, and the category is then assigned based on the range in which the score falls.

3.6.3 Composite Decision Score

The freshness score is one of the inputs used by the Decision Engine to generate the final status of the fruit. The Decision Engine combines the freshness score with the estimated remaining shelf life to produce a composite decision score.

The composite score is calculated as:

$$C=(S_{\text{freshness}} \times 0.6)+(D_{\text{norm}} \times 0.4)$$

where:

- C is the composite decision score
- $S_{freshness}$ is the freshness score from Model 2
- D_{norm} is the normalised remaining shelf life

The normalised remaining shelf life is calculated as:

$$Dnorm = \frac{\text{Days Remaining}}{\text{Base Shelf Life}}$$

$D_{norm} = \frac{\text{Days Remaining}}{\text{Base Shelf Life}}$

The value is constrained to the range 0 to 1.

The composite score combines two sources. The freshness score, weighted at 60%, represents the fruit's visual state. The normalised remaining shelf life, weighted at 40%, represents how much of the expected shelf life remains relative to the baseline.

The composite score is then mapped to one of four statuses:

Table 3.13: Composite Score to Status Mapping and Actions

Composite Score (C)	Status	Action
< 0.20	SPOILED	Discard

The sequence from prediction to final status is:

Predicted Class → Freshness Score → Shelf-Life Estimate → Normalised Days → Composite Score → Final Status

This process ensures that both the visual assessment of the fruit and its estimated remaining shelf life contribute to the final recommendation provided to the user.

3.7 Deployment

In this section the assembled working Fresco system is shown along with the request pipeline for a given user query to yield a result.

3.7.1 Deployed System Components

As a single, working application, the system comprises four components:

(i) a frontend; (ii) a backend; (iii) a database; and (iv) a model servingservice.

Frontend Fresco deployed as a Progressive Web App using Next.js and built with React.

Users interacting through a mobile browser install the app by adding the shortcut to their home-screen; an experience similar to native apps but still served over HTTP. Features include image uploads/captures for inspection, viewing scan reports, inventory management, and check storageability. The application communicates with the backend service using HTTP requests. Backend Built using FastAPI; the backend serves all client requests.

On receiving an image from the frontend, input is validated and the image sent on to the scan processing pipeline.

It returns structured responses to be rendered on the frontend via RESTful HTTP service endpoints including authentication, scan processing, inventory management, and referencing an input fruit (see Section 3.2.4) The service handles many requests concurrently via asynchronous requests. Database The system database is PostgreSQL for production and MySQL for

development. This is used to store the fruit index table, the user login and associated inventory and any scheduled user notifications.

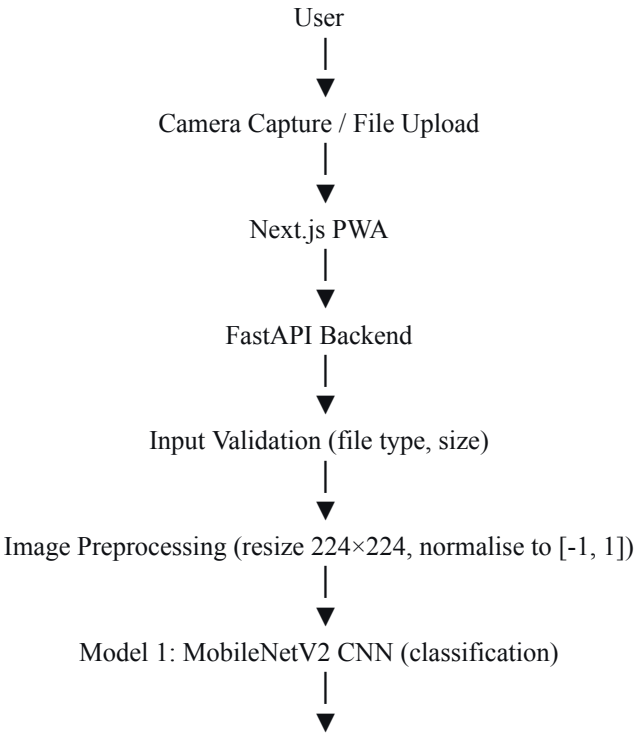
During the scan process the application will check with this database in order to reference an input fruit as well as upon receipt of new inventory items. Model Serving Once instantiated, the trained MobileNetV2 model (Section 3.2.1) is immediately loaded into memory. To initialize the computation graph, a single, dummy input is provided to obtain an output.

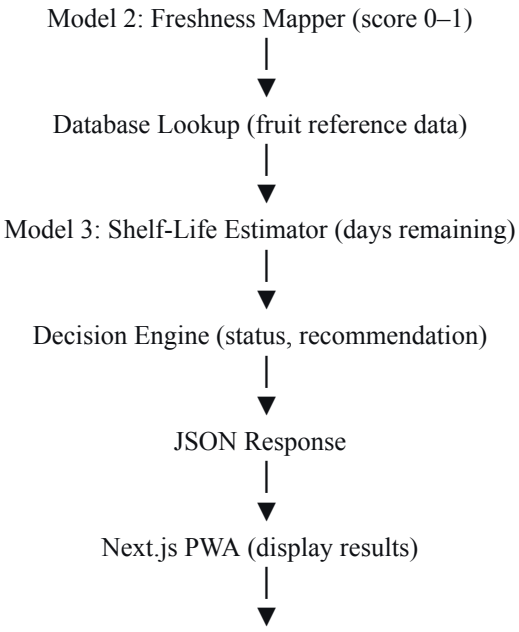
From this point on, the model object is held in memory within the server for its lifetime and as such does not need to be re-initialised.

3.7.2 Request and Response Flow

The complete flow of a scan request through the deployed system is shown below:

Request and Response Flow of the Deployed Fresco System





The user captures an image using the device camera or selects an existing image. The frontend sends the image to the backend through the scan API endpoint as multipart form data.

The backend validates the uploaded file by checking the file type and size. The image is then preprocessed by resizing it to 224 × 224 pixels and normalising the pixel values to the [-1, 1] range expected by the model.

Model 1, the MobileNetV2 classifier, analyses the preprocessed image and produces a predicted class label and confidence score. Model 2, the freshness mapper, converts the class label into a continuous freshness score between 0 and 1.

The backend queries the database to retrieve the fruit's reference data, including base shelf-life values, optimal temperature range, and ethylene flags. Model 3, the shelf-life

estimator, uses the freshness score and reference data to calculate estimated days remaining for room temperature, refrigerator, and freezer storage.

The decision engine combines the freshness score and normalised days remaining into a composite score, maps it to a status, and generates a recommendation string. Where applicable, an ethylene compatibility check is performed against the user's existing inventory.

The backend returns the complete result as a structured JSON response. The frontend displays the result to the user, including the status, estimated days remaining, recommended storage method, recommendation message, and ethylene advisory note where applicable. The user can then choose to save the scanned fruit to their personal inventory.

3.7.3 Deployment and Performance Characteristics

The main deployment and performance characteristics of the system are summarised in Table 3.6.

Table 3.14: Deployment and Performance Characteristics

Metric	Value	Description
Supported formats	JPEG, PNG, WebP	Accepted image formats

The inference latency of 200 to 500 milliseconds is measured after the model has been loaded and warmed up. The first request after server startup takes approximately 4 to 5 seconds due to the initialisation of the model's computation graph.

The backend is configured to handle concurrent requests through asynchronous request handling, with the model inference executed in a thread pool to avoid blocking other requests. Cross-origin requests are restricted to configured application origins.

CHAPTER FOUR: IMPLEMENTATION

IMPLEMENTATION

4.1 Introduction

In Chapter 3 we described Fresco's design: the architecture, database schema, API contracts, AI pipeline, frontend design, security model, and the tradeoffs that were consciously made in designing them. In this chapter we describe the realization of that design: the tools we employed, the development workflow, the sequence in which the modules were implemented, the concrete code organization that resulted, and the specific problems that arose and their solutions.

We have attempted to be very specific. Where we presented an idea in Chapter 3, here we name the files, present line counts, and actually trace the execution through the code. Where features appeared straightforward on paper, we have been up front about their complexity in this chapter. Where relevant, several short worked examples and the shelf life and decision engine formulas appear as well. A design is not credible until one can track through it a real input to determine how to compute the actual output.

The chapter is organised by the sequence in which the system was actually built. Section 4.2 describes the development environment. Section 4.3 and 4.4 provide details about the skeleton backend application and the database layer. Section 4.5 is by far the longest section in the chapter and we will spend it tracing down the AI pipeline module by module. Section 4.6 connects these together using the scan service. The UI is covered in 4.7, and the auth system in 4.8, and the PWA implementation in 4.9. We will finish the chapter by describing the primary development challenges in 4.10 and wrap up in 4.11.

4.2 DEVELOPMENT ENVIRONMENT AND TOOLS

All of the development and the vast majority of system integration work took place locally on personal computers (both Windows and macOS). Visual Studio Code was the Integrated Development Environment (IDE) used on all machines and for virtually all local development work. Training was done on the Google Colaboratory service, because model training was particularly compute intensive, and our local machines did not contain dedicated GPUs to

accelerate the process.

Version control, using a private GitHub repository, was used to track our progress. The development stack and (where applicable) version are given below:

Table 4.1: Development tools and environment

Category	Tool	Version	Purpose
Language (backend)	Python	3.12	Backend runtime
Language (frontend)	TypeScript	5.x	Frontend type-safe development
Web framework	FastAPI	0.115+	Backend HTTP layer
ASGI server	Uvicorn	0.30+	Application server for FastAPI
Frontend framework	Next.js	16.2.1	Frontend application framework
UI library	React	19.2.4	Component model
Styling	Tailwind CSS	v4	Utility-first styling
ORM	SQLAlchemy	2.0	Database abstraction with async support
Database (dev)	MySQL	8.x	Local development database
Database (prod)	PostgreSQL	16.x	Production database on Render
Database (fallback)	SQLite	3.x	Zero-config testing fallback
Validation	Pydantic	2.x	Request and response schemas
ML framework	TensorFlow / Keras	2.16	Model definition and training
Model architecture	MobileNetV2	(ImageNet pretrained)	Transfer learning base
Image processing	Pillow	10.x	Image loading and preprocessing
Cryptography	passlib (bcrypt) + hashlib	1.7 / stdlib	Password hashing
Tokens	python-jose	3.x	JWT issuance and verification

Category	Tool	Version	Purpose
Training environment	Google Colab (T4 GPU)	—	GPU-accelerated model training
Editor	VS Code	1.94+	Primary development environment
Version control	Git + GitHub	—	Source control and collaboration
Backend deployment	Render	—	Production hosting target
Frontend deployment	Vercel	—	PWA hosting target

Choosing to use 3.12 over the brand new 3.13 was intentional. Although official TensorFlow wheels supported 3.13 on many platforms back then, this support was not yet across-the-board rock solid. Thus 3.12 serves as our solid baseline where we mix all the cool new language features (nicer error messages, type parameters syntax, mature asyncio API) with full support from all of our libraries. Our frontend tooling uses node.js 20.x. The repo contained two top-level directories; backend and frontend, sharing one .git repository, but separated by distinct .gitignore files.

My daily editor for both backend and frontend development was VS Code with extensions such as the official Python extension, Pylance, ESLint, Prettier, and Tailwind CSS IntelliSense. The repo contained two top-level directories; backend and frontend, sharing one .git repository, but separated by distinct .gitignore files.

4.3 Backend Application Skeleton

Our backend is in the directory backend/app. The main entry point to that module is backend/app/main.py (185 lines), which had three major responsibilities: creating the FastAPI application, mounting middleware on it, and handling the application's startup and shutdown life cycle events.

4.3.1 The FastAPI Application

To create a FastAPI application, you create an app object and then mount routers on that object. Here we create the app at module load time, then mount four routers on the app under distinct

URL prefixes: /auth, /fruits, /scan and /inventory. Last but not least we mount the static files handler to the /uploads path which makes the scanned images we store in the scan service available for us to load back up in the frontend at some point in the future.

CORS is configured with FastAPI's CORSMiddleware. In development we allowed specific origins (http://localhost:3000, http://127.0.0.1:3000), although in production we read from the ALLOWED_ORIGINS environment variable which allows us to point the same backend code to any frontend deployment without needing modifications. We allowed all HTTP methods and all headers on all allowed origins, and allowed credentials to permit the JWT cookie our frontend sends to carry over on cross-origin requests.

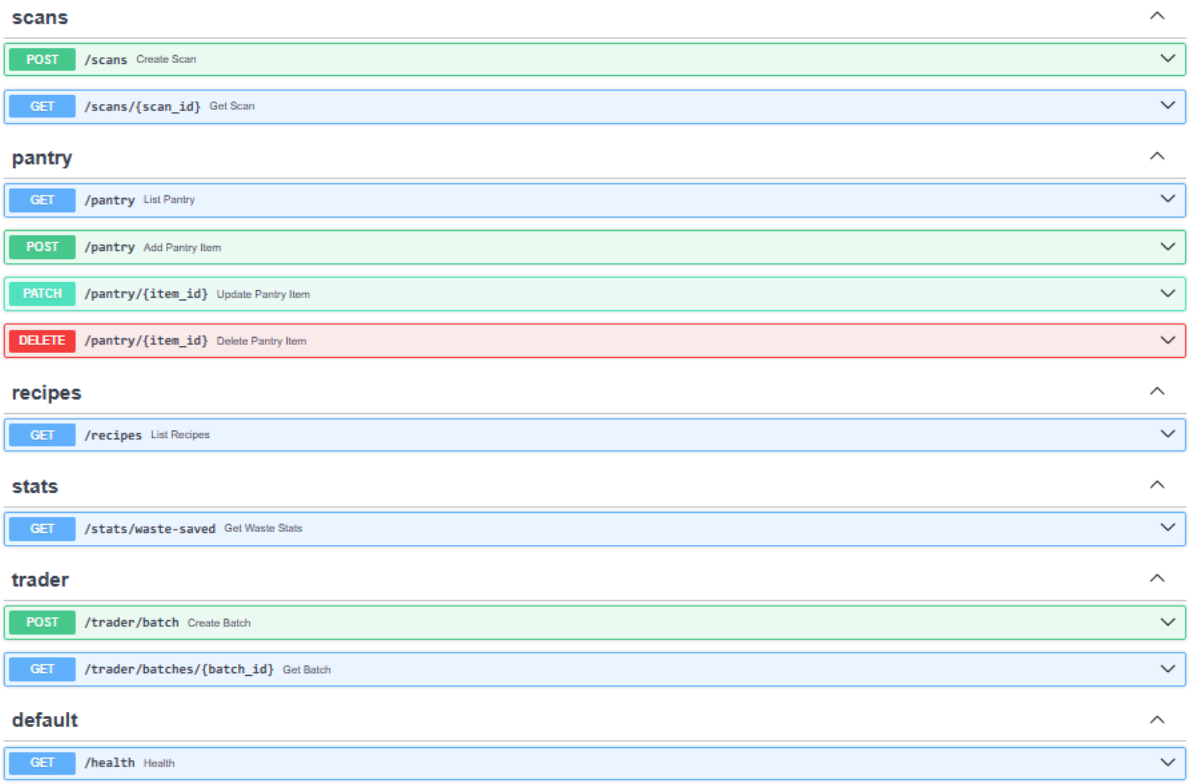


Figure 4.1: *Auto-generated OpenAPI (Swagger UI) documentation for the Fresco backend, showing the scans, pantry, recipes, stats, trader, and health API endpoints with their HTTP methods and route paths.*

4.3.2 The Application Lifespan

The lifespan handler is a method that runs in parallel with the application and allows you to run two main types of processes (startup and shutdown) using the @asynccontextmanager decorator.

I had three items to take care of for startup. I created tables based on ORM model if the tables were not existing, I populated fruits table based on the 42 manually entered fruit records described in Section 4.4.2 if there was no fruit entry and I loaded and initialized (warm up) the model in memory so that the cost of inference would not be sustained by the first consumer request.

Warm up is critical here. When Tensorflow loads a model from its disk and applies inference for the first time, it undertakes a set of heavy computational processes such as computation graph compilation, kernels JIT compilation based on CPU instructions and memory allocation. If a warm up does not run before first user request of scanning the fruit-to-scan, a user might get the difference between inference time ~200ms and ~4/5s so in order to omit this drawback, I ran an inferential test prediction on a black matrix 2242243 in the startup procedure and that the user does not pay any extra charge to load the ML model.

4.3.3 Pydantic settings and Configuration

The backend configurations are set under the config.py file (103 lines). It is a class which inherits the pedantic base setting. It is composed of 14 fields such as the URL for the DB, JWT's secret and expiry time, the allowed domain names for cors request, max allowed file size, the directory for file upload, model path and the DEBUG variable. The 14 fields have default values which will be overridden by an environment variable if available. Each of the configurations is validated at launch time thanks to Pydantic's strong typing.

In terms of local development, all environment variables defined in this file will be loaded directly from a local .env file in the root directory for BaseSettings class to handle it. On Render deployment, it's similar using Render's environment setting available for applications. Indeed, both secure the usage of secret information because you won't make comments on this information.

I added custom validation in the DEBUG fields when I faced a deployment problem. Usually, it's just an boolean: DEBUG=True, and since in python 'false string' can be interpreted as a true value (because the string is not empty) then when you write DEBUG=release or DEBUG=prod or DEBUG=production, python evaluates them as True. To avoid this misleading behavior, the validation has been customized and only 'dev' or 'development' would lead to set the debug value

to True and 'release', 'prod', 'production' set to false. Also the values from environment variables would be standardized into these 2 states at configuration load time.

4.4 Database Implementation

4.4.1 Async Engine and URL normalization

The database tier is the one in charge in 'database.py'(90 lines of code). It initializes an async SQLAlchemy engine and returns a session factory. What's important is its function that lets you get a valid DB URL. The driver part is very strict with SQLAlchemy, by default in mysql: mysql: mysql:// user:mysql://user:pass@host/db, a synchronously running driver is used. And this is not compatible with the async engine which requires you to specify the driver explicitly as below for the two possible DBs in this project; mysql+aiomysql://... And postgresql+asyncpg://... .
As I wrote previously, most providers would offer a connection string based on synchronous syntax i.e for render Postgres add-on, postgres://...; and the deployer had no choice than to rewrite it carefully by hand for the system to be async working, however that would create inconvenience for the programmer. My solution is to use a URL normalization function; preparedatabaseurl(): It detects the DB URL scheme and replaces it with the correct async driver URL before it is used by the SQLAlchemy. It handled SQLite's exception too for a correct connection using sqlite+aiosqlite://... And the connect-arguments dictate variation. The session factory part is embedded inside an async session maker which generates async-session when it's needed. Databases accessing routes take their sessions directly through the FastAPI depend function get db which produces an async-session iterator guaranteeing to close it no matter what the process was.

4.4.2 ORM Model and Seed Data

The four ORM Models are implemented in 'models/schema.py' (164 lines of code): they are the ones named Fruit, User, UserInventory and Notification and, as mentioned in the Chapter Three of this thesis document, each of them have their corresponding table in DB. Relationships between models are defined bidirectionally using Mapped and relationship decorators (SQLAlchemy 2.0). CASCADE-on-delete is set up at FK constraint level and at ORM level.

The redundancy comes from the database level guarantee.

This last means that a deletion from one end must cascade the deletion of all other dependencies, without needing for the DB's referential integrity to enforce it (as this may fail on some configurations). The seed data is loaded in 'seed_data.py' which is the largest back-end file of my project (617 lines of code). The data are constituted as a Python List of 42 dictionaries. Each dict represents a fruit having all details as required on the reference information such as sub category, its shelf life in the 3 methods of preservation and ethylene producer/susceptible flags.

All data were acquired from food reference guides, mainlyUSDA Postharvest Handling Guidelines , ethyne post-harvest biology.

I also added a basic authentication/identification for a default testing user that gets into the database if user 'admin'does not exist to ease my local development. These 42 items are loaded on application startup only if there is no record in the fruits' table, which makes data loading redundant at restart/deployment/migrating, for it also guarantees on loading test data.

4.4.3 Pydantic Scheme on the API Border

The Request and response schemes are defined using the schemas/schemas.py file(312 lines of code). They can generate a Json schema which is then used by FastAPI, when writing the OpenAPI page of the app (/docs). The most powerful aspect of Pydantic schema is that it handles validation at API border, as a consequence an invalid request body won't be directly sent to the route function to face some potentially failed processing and thus throw up. An invalid body, e.g, a json body like '{"name": 42}' which was supposed to take a String for 'name', will be rejected with a 422 error and an accurate message of the failing field and a hint about the failed validation, all auto-generated.
The ScanResponse scheme took particular attention. The three-tier approach as shown in Section 3.4.2 of the document is coded as 3 inner Pydantic models namely, ScanDecision (consumers face view: score and shelf life of the fruits in the 3 differents conditions, recommendations on what to do), ScanDetail (for internal view of the results: predicted fruit details and a classification confidence), and ScanContext (which contains in additional any advisory for the fruit like ethylene emission and compatibility concerns with user-listed inventory). All of those models were defined to generate a nice documentation on the API page and also allow typed fetching for the front end developers.

4.5 The AI Pipeline: Module by Module

The AI components live under the backend/app/ai/. There are six files in this directory, totalling approximately 965 lines of Python. The rest of this section walks you through each module as it's executed by a scan request.

4.5.1 The Classifier Module

The classifier lives in classifier.py (131 lines) and performs three tasks: loads the trained MobileNetV2 model as a lazy singleton from disk, preprocesses an uploaded JPEG to the [-1, 1]-normalised 224x224x3 tensor the model requires, and provides the predict() function that executes inference and post-processes the results. The model file itself is an artefact of the training process below (4.5.1.1), stored as frescomodel.keras which, at 11.6 megabytes, is the result of MobileNetV2's parameter efficiency. It’s loaded at the time the getmodel() accessor function is called, and the trained model is cached at the module level for reuse with future calls; on average over an entire scan request, loading was estimated at 2-seconds apiece, while this module uses a singleton pattern to defer the cost to a one-time startup activity.

4.5.1.1 Model Training on Google Colab

The model training phase occurred within Google Colab’s free T4 GPU runtime. A Jupyter notebook was used, loading both the Fruits-360 dataset [1] and a customized selection of additional fresh and stale fruit samples; data organised into 14 relevant classes; and transfer learning with the MobileNetV2 [2] architecture. Two phases were conducted on the MobileNetV2 base: first, for 10 epochs with a learning rate of 1x10-3 the training layer was fit, and in a second phase the top-30 layers were unfrozen and trained with a learning rate of 1x10-5 for an additional 15 epochs.

Categorical cross-entropy served as the objective function, the Adam optimisation algorithm provided gradients with respect to the parameters, and data was provided to training in batches of 32.

Initially 64, the batch size was reduced due to multiple out-of-memory exceptions caused by the GPGPU usage of activation tensors of MobileNetV2 layers being higher than the prior-with-frozen-layers phase had led expectations to predict.

For training, images were augmented via random horizontal flips, rotation through angles up to 15 degrees, and changes to image brightness (20%) and zoom level (10%). This data augmentation policy was established in order to simulate effects in real consumer photographs - that some pictures will be at various angles and in less ideal lighting - without creating images with characteristics unlikely to arise in practise. 80% of the curated and processed data was retained as training set, a 10% portion as validation set, and a 10% held-out test set. Total training time for the model was around 47 minutes.

The final model achieved classification validation accuracies of 91.4% over the 14 classes, with an apparent ceiling reached at epoch 18 by the classification layer and only marginal improvements thereafter.

The final accuracy for the held-out test set is reported elsewhere.

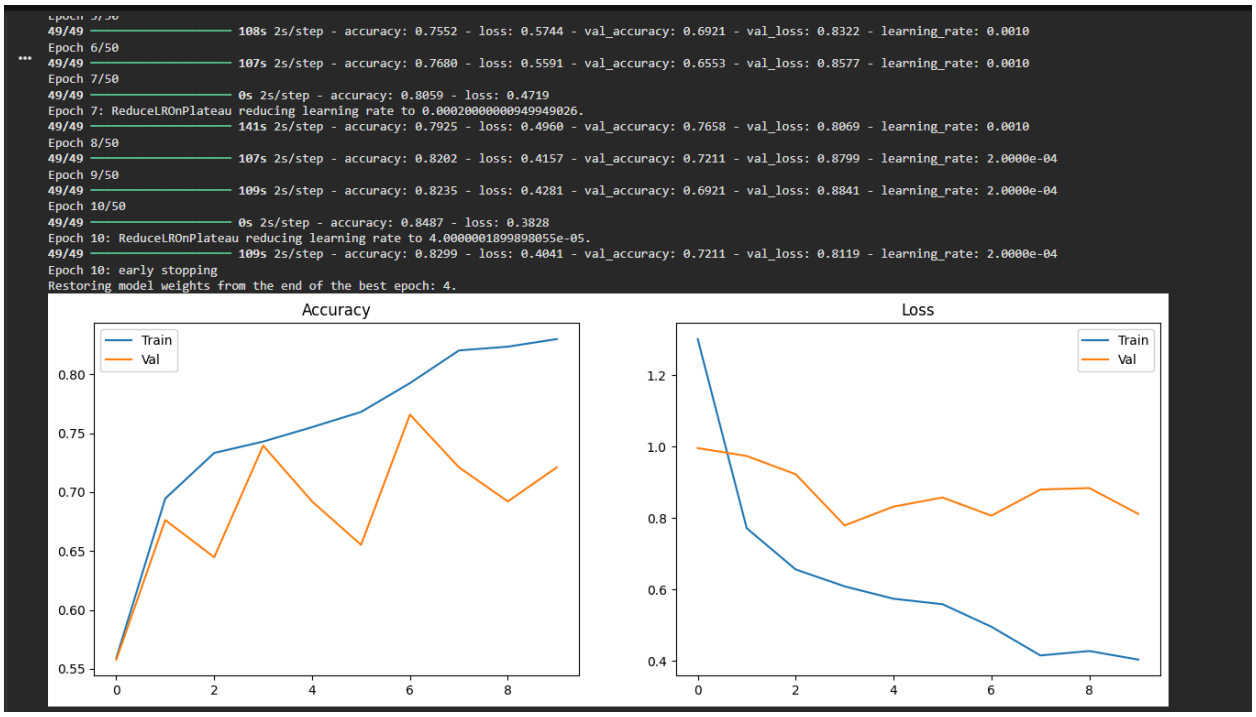


Figure 4.2: *MobileNetV2 model training and validation accuracy (left) and loss (right) curves across epochs. The model was trained on Google Colab's T4 GPU over two phases: frozen-base classification head training, followed by selective fine-tuning of the top convolutional layers.*

4.5.1.2 The 14 Classes and an Honest Note on Two of Them

The classifier has 14 output classes, corresponding to the four fruit types, Apple, Banana, Carrot, and Tomato, for the several timeliness ranges and one 'Expired' class for all four fruits. Class labeling follows that the publicly available dataset from which it was inherited was originally put together for use as input to a broader food-quality research endeavor, and not necessarily a focused retail tool.

Of the 4 fruit types, only Apple and Banana exist on our curated 42-fruit reference database, for production purposes. Carrot and Tomato exist merely as training-set artifacts-they may be predicted by the model, but our database lookup at step 3 of the scan process will yield no result. The actual implication is the following: if the model predicts Carrot or Tomato with significant conviction (something that happens very infrequently for actual consumer product images, since they fall out of the distribution), the system automatically flows to a path involving manual fruit selection. Although we considered adding a step which retooled the model's head and removed those classes, we determined that they contributed to enough variety among possible features during training to offset the negative cost that they present. Their role then is one of general augmentation rather than as a class supported by a production deployment.

This step illustrates that the correct engineering choices to solve certain problems might appear awkward initially, but solve that problem more robustly. Including the two additional classes causes the model to have additional non-target items pushed against Apple and Banana in the feature-space-and we are therefore better at separating the fruit categories we wish to use.

4.5.1.3 Preprocessing and Inference

The preprocessing function willLoad the JPEG file via python's Image Library; convert image to RGB color space (this will take corner cases for greyscale one-channel uploads and four-channel images from RGBA) ; resize images to 224x224 and apply bilinear interpolation ; convert image to a numpy float32 array; transform pixels using the formula: (pixel/127.5)-1.0); reshape the array adding a leading 'batch' dimension; yield the output. The entire transformation sequence

matches the input preprocessing done during the ImageNet pre-training performed before any fine tuning was completed-and this is essential for it to leverage features learned through transfer learning during inference.

Inference on the model is enclosed within asyncio.to_thread() so the synchronous call to TensorFlow will not block the FastAPI web-server. While the model is processing we can attend to another request on our web server. This is important for the single scan inference which, while quick, is the slowest task, and starving its loop would cause our every concurrent request to slow down.

We have Two postprocessing steps applied after the raw inference. First, we forcibly set the class probability for the "Expired" class to zero and then take the argmax; this modification comes from observations during evaluation that this class was being heavily favored given slight over-sampling in the training set and its broad interpretation. We made this modification at effectively no added cost using a single line in Numpy. Second, we take argmax over the now-13 classes and then use a regular expression in Python to extract the fruit name from the predicted class, omitting the (N-M) attribute and title casing the word.

The Freshness Factor is also calculated from the classification. The class itself contains the estimated timeliness (banana freshness score=(5-10) for a banana aged 5-10 days, for example), and our freshness mapping looks up a score from that class, corresponding to the range 0-1; the early classes represent timeliness 0.85-1.00 (fresh) through the final classes representing timeliness 0.40-0.59 (decaying)-we manually calibrated this mapping using values for timeliness gathered from the labelled data set.

4.5.2 The Shelf-Life Estimator

This code, housed in estimator.py and 271 lines long, is arguably the most algorithmically rich piece of the AI Pipeline. Nevertheless, it takes two inputs only: a reference fruit object and a single score indicating how fresh it is. As discussed previously, it operates by composing three independently derived metrics together into a final answer for freshness according to this equation:Estimated Days Remaining = Base Shelf Life(Freshness score)^1.5(Temperature Factor)(Ripeness Factor)It computes this value for each of the three storage method configurations, and our decision engine later selects the recommended path.

4.5.2.1 Example Calculation

A real world input to the estimator: a banana fresh-ness score of 0.72 derived from a Banana(5-10) prediction (e.g.). From our seeded database row for bananas, an input banana has reference data as follows: room-temp baseline=5 days, refrig baseline=7 days, freezer baseline=60 days; its optimal temperature for storage is in the 13C-15C range.

Our Freshness Factor can be calculated by raising the 0.72 to the power of 1.5 giving 0.611 roughly. The exponential represents that food quality degradation is worse in its last stages. The fruit is past more than 28% of its useful life as shown by a 72% freshness score. The Ripeness Factor is found based on our freshness score 0.72; it is in the nearly ripe (0.70 score<0.85) category which has a multiplier 0.85.

The Temperature Factor is conditional on the selected storage technique. For a room temperature-22C storage scenario, the stored temperature falls outside of the optimum temperature of 13C-15C range by 7C and would typically entail a 14% loss-in useful freshness (since we apply 2%/°C degradation). The factor here is 0.86. For refrigerator storage (4C); the temperature is outside the optimum 9C below that optimum range which would typically apply 18% penalty. Bananas can suffer chilling-injury below 12C, so 2%/°C is still applied correctly giving a 0.82 Temperature Factor. A freezer storage temperature of -18C entails a fixed 0.95 factor, representing the minor deterioration upon freezing and thawing.

Composing the three factors against each storage method's baseline:

Storage method	Base × Freshness^1.5 × Temp × Ripeness	Days remaining
Room temperature	5 × 0.611 × 0.86 × 0.85	~2.2 days
Refrigerator	7 × 0.611 × 0.82 × 0.85	~3.0 days
Freezer	60 × 0.611 × 0.95 × 0.85	~29.6 days

These 3 numbers are put into the decision engine, which chooses which of the storages that is furthest of its predicted end of shelf-life (in this case, the freezer). It chooses 29.6 days left (it will tell the user that while freezing does change texture of bananas and thus better fit cooking rather than raw use, it offers longest available shelf life) 4.5.2.2 Implementation Notes The estimator function takes in the fruit and it predicted freshness

score, and outputs a dictionary where keys represent the possible storages and the value represents the predicted days until it perishes. The variables for temperature are constant throughout at, room 22 o C, refrigerator 4o C, and freezer -18 o C.
It uses a module level list that stores the temperature-dependent multiplier at different parts depending on fruit state (represented in form of) tuples. 5(a) Fruit ripening stage based ripeness thresholds list (fruit etc as constants)
The module level list contains (threshold, multiplier) tuples that will estimate the ripening-based factor. The steps here are descending, therefore the multiplier only kicks in below that threshold. In this situation, the stages greater than 0.8 and 0.6 bring about lower bounded temperature-based factors. The limits below bring a factor, at greater than 0.4, greater than 0.2 and greater than 0 and it causes lower level multiplier effects. We place an upper limit so that a temperature-based factor can never get below the 0.5 point so that the overall shelf-life isn't drastically small.

These low factors weren't considered useful for the user and potentially negative therefore 0.5 was put in as the lowest temperature factor so if optimum storage conditions aren't present for a fruit, say, a banana at 22C rather than -18C, the shelf life won't be drastically lowered although it does have a decreased shelf life..

4.5.3 The Decision Engine

The decision engine combines data from the classifier and the estimator along with any compatibility rules to form the final verdict. Located in 'decision_engine.py' (245 lines). Once the result has been generated from all above processes and outputted as a decision 'Verdict', it contains the necessary data - user-facing status, overall confidence value, recommendation string, recommended storage type, and any note about ethylene gas emission.

The way these results are combined can be seen as basic but crucially important. The verdict cannot be changed by any of the processes previously described and remains as it is. It consists of a score composed of the 'freshness' score weighted by 0.6 and a calculated normalized remaining days value weighted by 0.4:

compositescore freshness(0.6) + normalizedremainingdays(0.4)

, where normalized remaining days are [days remaining stored for that type] / [days baseline for that storage method] Each is normalized. They are then weighted as described above so that a useful number can be produced between 0 and 1 that doesn't rely on specifics of storage.

Normalisation allows us to compare each of the factors without fruit-specific comparison. E.g for strawberries 3 days left until perishable is extremely low whereas for apples 3 days is not that big a deal. These 3 days are useful but cannot be directly compared. They are therefore normalized to a percentage that also gives an indication within a range of 0-1 that can then be compared with other scores.
4.5.3.1 A worked example, revisited The banana that was mentioned before is the

primary example of the Decision Engine working-with a maximum days of 29.6 in freezer, and 60 days for optimum:

normalizedremainingdays 29.6 60 = 0.493. The use of 0.493 is taken into the recommendation with the weighting value same value due to it not being greater than 1 and lower than 0 for the comparison.

Compositescore [0.72 0.6] + [0.493 0.4] = 0.432 + 0.197 = 0.629.

The conditions aren't analysed in any order, only based upon the final combined score. With it below 0.70(fresh) and greater than 0.40(Eat soon) Status = Eat soon. The recommendation string for this banana is "your banana needs to be eaten in the next 2-3 days For further storage options in this situation try the freezer",

4.5.3.2 Recommendation strings and ethylene notes The recommendation string is constructed from a template per status. This takes the name of the fruit and puts it along with the number of days left and the suggested storage method. These are manually generated, with the instructions about the urgency and helpfulness of the string considered during generation: in certain status points like EATTODAY or SPOILED the string needs to be extremely direct, however in others the user requires encouragement as much as possible. The Ethylene note is done last after determination of status and utilizes the ethylene producer and isethylene_sensitive values.

A fruit that is both (like a banana) has a paragraph containing that information; solely producer/sensitive fruits only contain info pertinent to those aspects of the plant's physiology. A fruit with none is only just not recommended as a strategy and having no special note.

The intention behind these less verbose comments on ethylene are to decrease the payload size of each result.

4.5.4 The Ethylene Compatibility Engine

The compatibility engine is part of the compatibility.py python file, which also makes up 208 lines of code. It comprises 2 components, namely; a load time building up of a graph of incompatibles, and the comparison of different components pairwise at request time.

4.5.4.1 Graph construction. It first sets up two frozenset constants which list the incompatible parts, namely: ETHYLENE\PRODUCERS and ETHYLENE\SENSITIVE. ETHYLENE\PRODUCERS are all the fruits identified to increase rate of ripening; Apple, Apricot, Avocado, Banana, Cantaloupe, Fig, Honeydew, Kiwi, Mango, Nectarine, Papaya, Passionfruit, Peach, Pear, Persimmon, Plantain, Plum and Tomato (with this being the only non-fruit that needs to be included due to its high yield), and ETHYLENE\SENSITIVE are all the fruits identified to be sensitive to ethylene gas; Apple, Banana, Blackberry, Blueberry, Cherry, Cranberry, Date, Grape, Grapefruit, Guava, Kiwi, Lemon, Lime, Lychee, Mandarin, Mango, Orange, Pear, Pineapple, Pomegranate, Raspberry, Soursop, Star apple and Strawberry. Intersection 5 contains all fruits that are found to emit ethylene AND are also sensitive to it: Apple, Banana, Kiwi, Mango and Pear; with it fitting directly in with what we know about a multitude of fruits that these naturally occur together in many common food types.

The graph of incompatiables (represented in terms of an adjacency list) is created once when the program is loading.

4.5.4.2 Pairwise

Checking If a user inputs fruit-names via the /api/fruits/compatibility endpoint the procedure is as follows. Firstly, the provided names are normalized to TitleCase, duplicate names are collapsed (retaining original order), so "BanANA" "apple" "BANANA" is treated the same as "Banana" "apple". Secondly, iterate through all pairs of fruits (unordered), consult the conflict graph for eachPair to determine if it[either][any] of the fruits in the pair (is one of the association keys, connected with an edge to[or from] the other?

If it is, create a pairwise conflict and add it to the result list.

Finally, for all those pairs that are associated with a conflict, generate a user-friendly, human readable report of the conflict, containing both names and an accurate description (briefly), of the potential damage (as: 'can significantly acceleratethe ripeningand spoilageofnearby...'). The entire comparison is performed in O (N) with N being the number of Fruits, it's around 190 comparative tests for a typical selection of common items, and takes milliseconds to run. This needs to be resilient to nothing more. It all is rounded off by a final grouping round that sorts the Fruits into two lists - 'ethylene-emitters' and 'others', simple yet brutally effective advice... 'things on the counter, those in the drawer' (to paraphrase the UI advice).

4.6 The Scan Service Pipeline The services layer is encapsulated in a class named ScanService residing in the services/scan_service.py (214 lines). Its job is to manage and orchestrate the seven stages outlined in Sec 3.5.1.

It calls each module sequentially, stitching together a unified answer from its various results. In an effort to ease debugging, the scan-service follows the execution flow top to bottom as a clear, bottom-top dependency view can confuse and distract. The ScanService class utilizes a custom, centralized error-handling mechanism.

When a pipeline module detects an error, it wraps any caught exception into a standard ScanError custom exception, which also holds an HTTP status code alongside a human-readable description of the fault. Any component of the pipeline that may fail is surrounded by a try-catch-exception block, which transforms any caught exceptions into such a ScanError object with its appropriate status and message, preventing a leaked internal 500 error by virtue of its being intercepted by the external Scan endpoint which translates it back to a proper HTTP response. The cleanup of an image upon a failed scan is centrally handled by wrapping the entire content of the scan-function within a try-finally block.

If, after an image has been successfully written to disk, any step subsequent to that one should fail, the finally block will cleanly delete the residual image, thus avoiding a cluttered uploads folder after unsuccessful scans.

The two step fruit name lookup discussed in the Sec 3 is, as mentioned in the explanation, two subsequent SQLqueries. An exact case insensitive matchis performedfirst, via func.lower(), then, if no matches are found, a case insensitive, wild carded LIKE operator is used for a substring search, and then, again, only if the first two do not yield any results, the system will resort to a pattern match with wildcards in the front. In case none of the searches yield a fruit name thefunctionwillRaise a ScanError with a 404 HTTP statusand a descriptive message indicating the searched fruit name and lack thereof in the reference data. This will signal the frontend to offer the choice to manually add this fruit. 4.7 Frontend Implementation The frontend of this project is developed using Next.js v16.2.1 (App Router), React v19.2.4, and TypeScript; and is housed in the frontend/ directory of the monorepo.

The frontend architecture features eight dedicated route directories under the app/ directory for distinct views, twenty-eight custom components distributed across four modular directories within the components/ folder, a consolidated API client exposed via lib/api/index.ts, an authentication context defined in lib/auth-context.tsx, and Progressive Web App (PWA) assets such as public/manifest.json and public/sw.js.

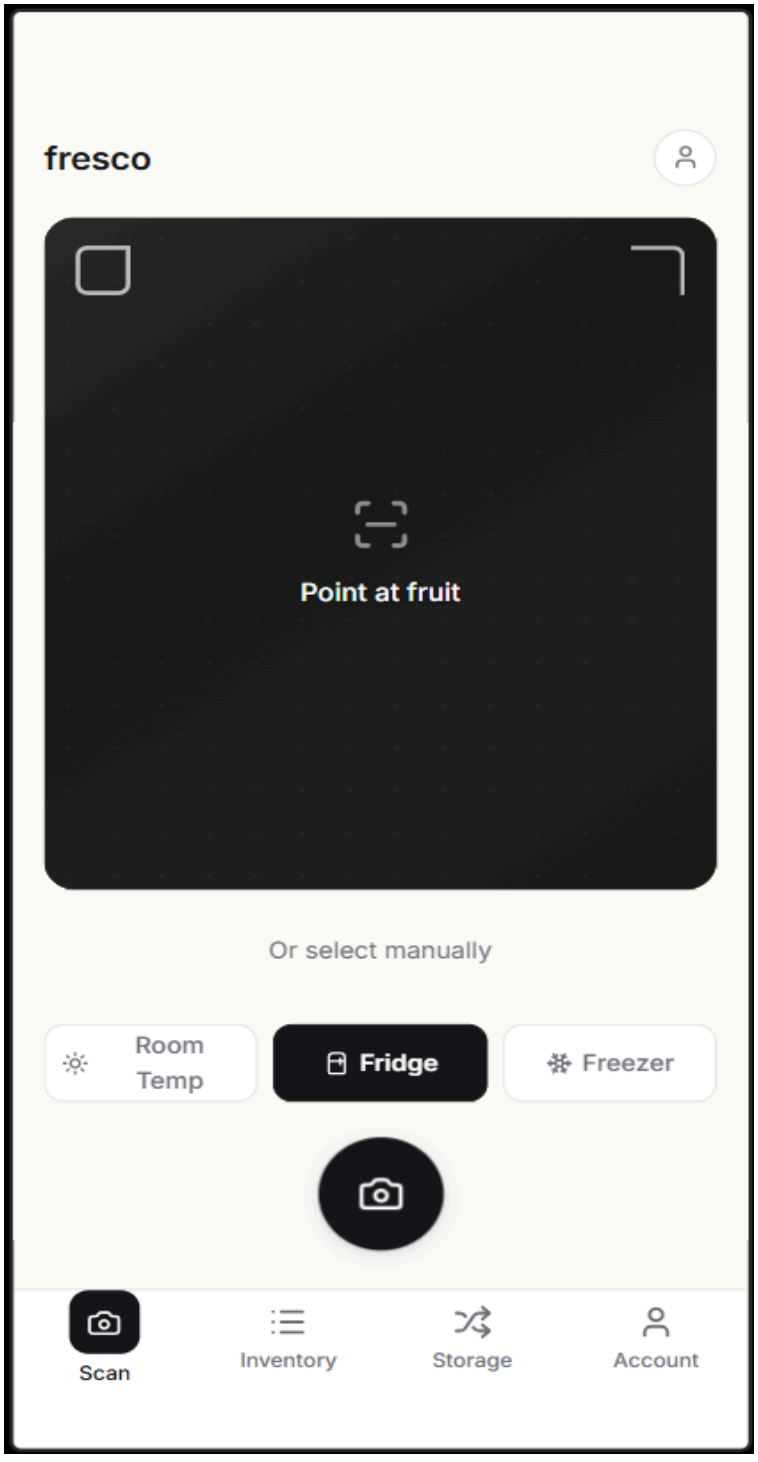


Figure 4.3: *The Fresco Scan screen showing the live camera viewport in active mode, the 'Point at fruit' targeting instruction, and the three storage-method selectors (Room Temp, Fridge, Freezer) the user picks before triggering the scan.*

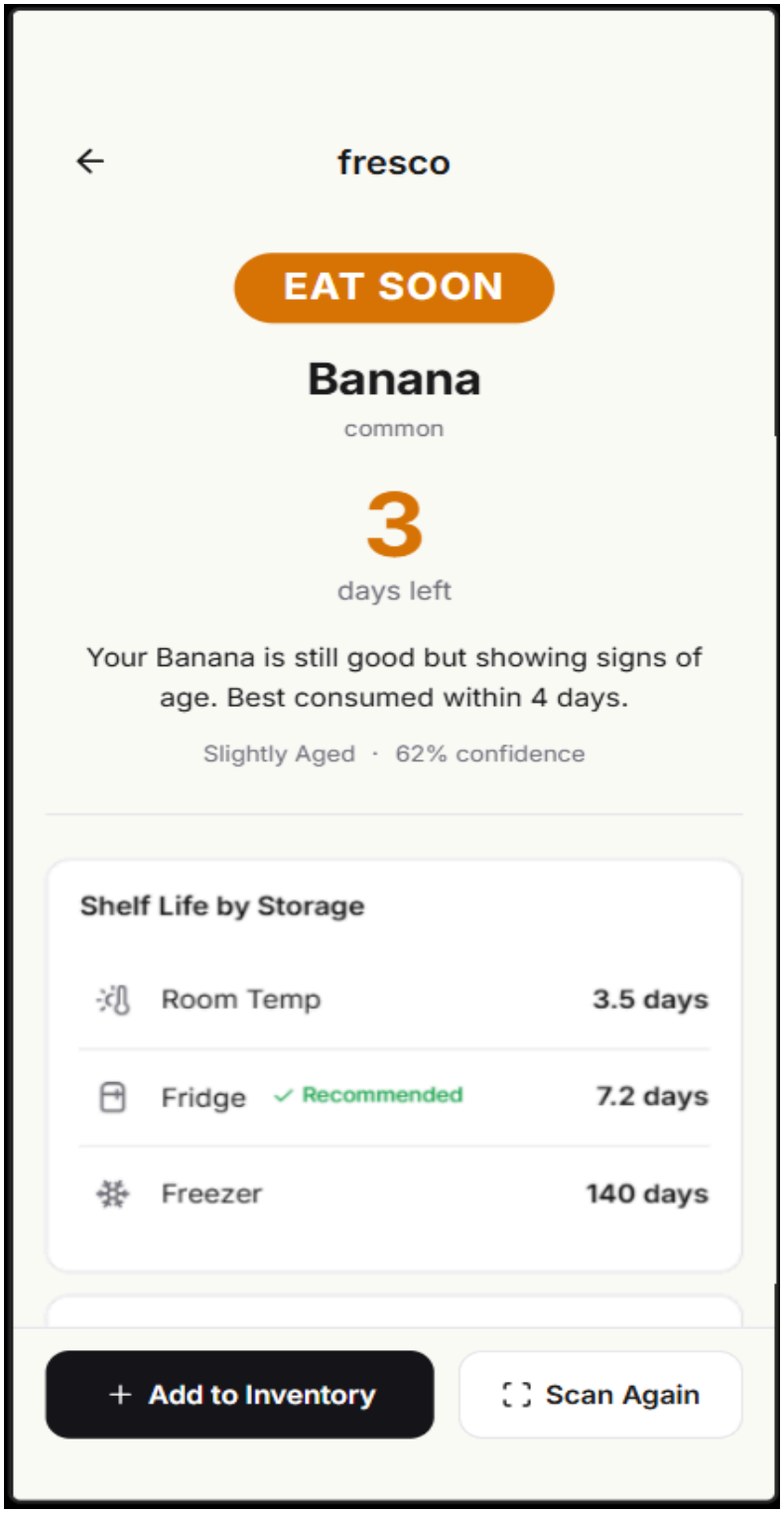


Figure 4.4: *The Fresco Results screen displaying an EAT SOON verdict for a Banana scan: freshness label 'Slightly Aged' at 62% confidence, 3 days remaining, and a shelf-life breakdown showing Room Temp 3.5 days, Fridge 7.2 days (Recommended), and Freezer 140 days.*

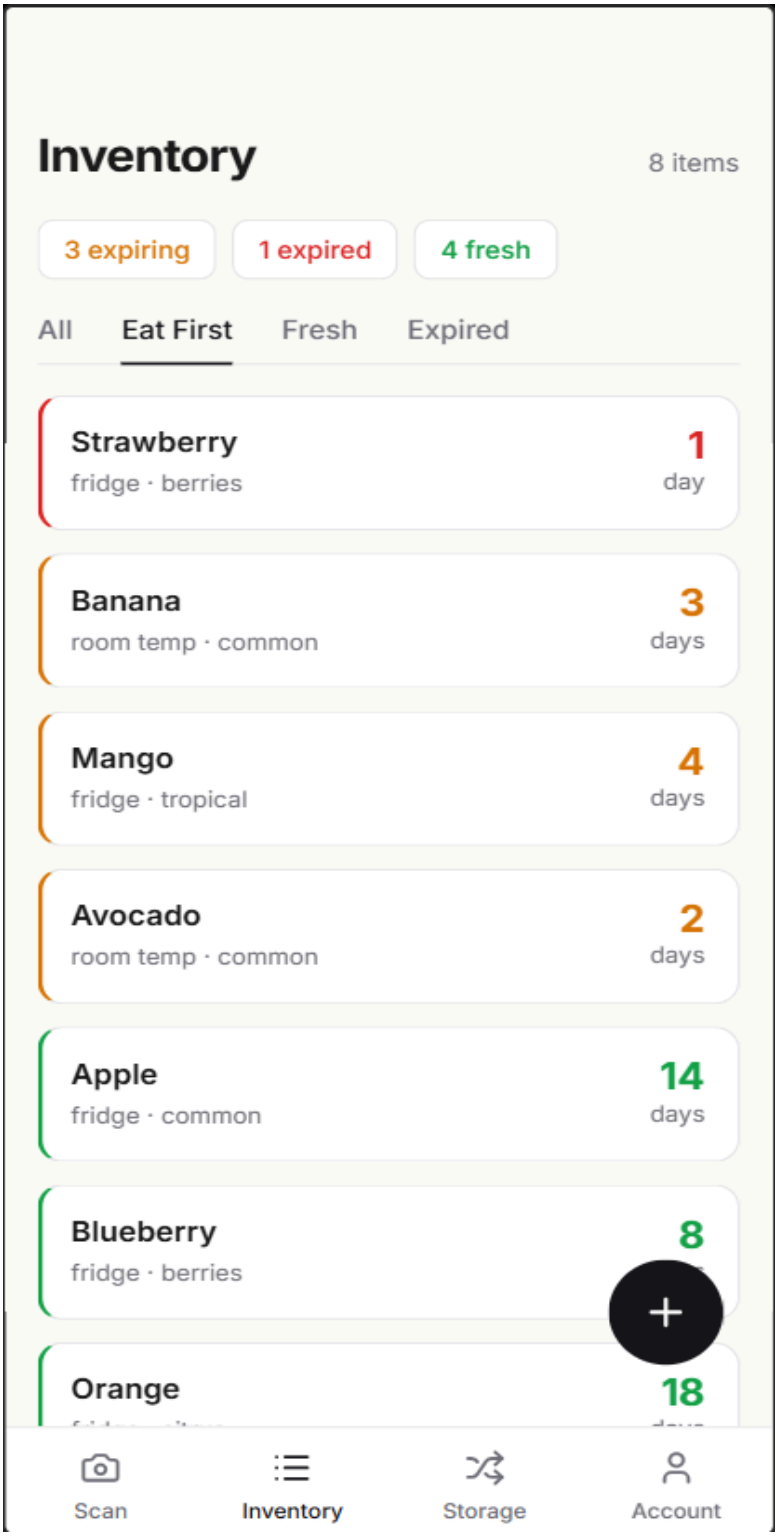


Figure 4.5: *The Fresco Inventory screen listing eight tracked fruit items sorted by urgency ('Eat First' view). Colour-coded day indicators range from red (Strawberry, 1 day) through amber (Banana 3 days, Mango 4 days) to green (Apple 14 days, Orange 18 days), reflecting the multi-factor shelf-life estimates.*

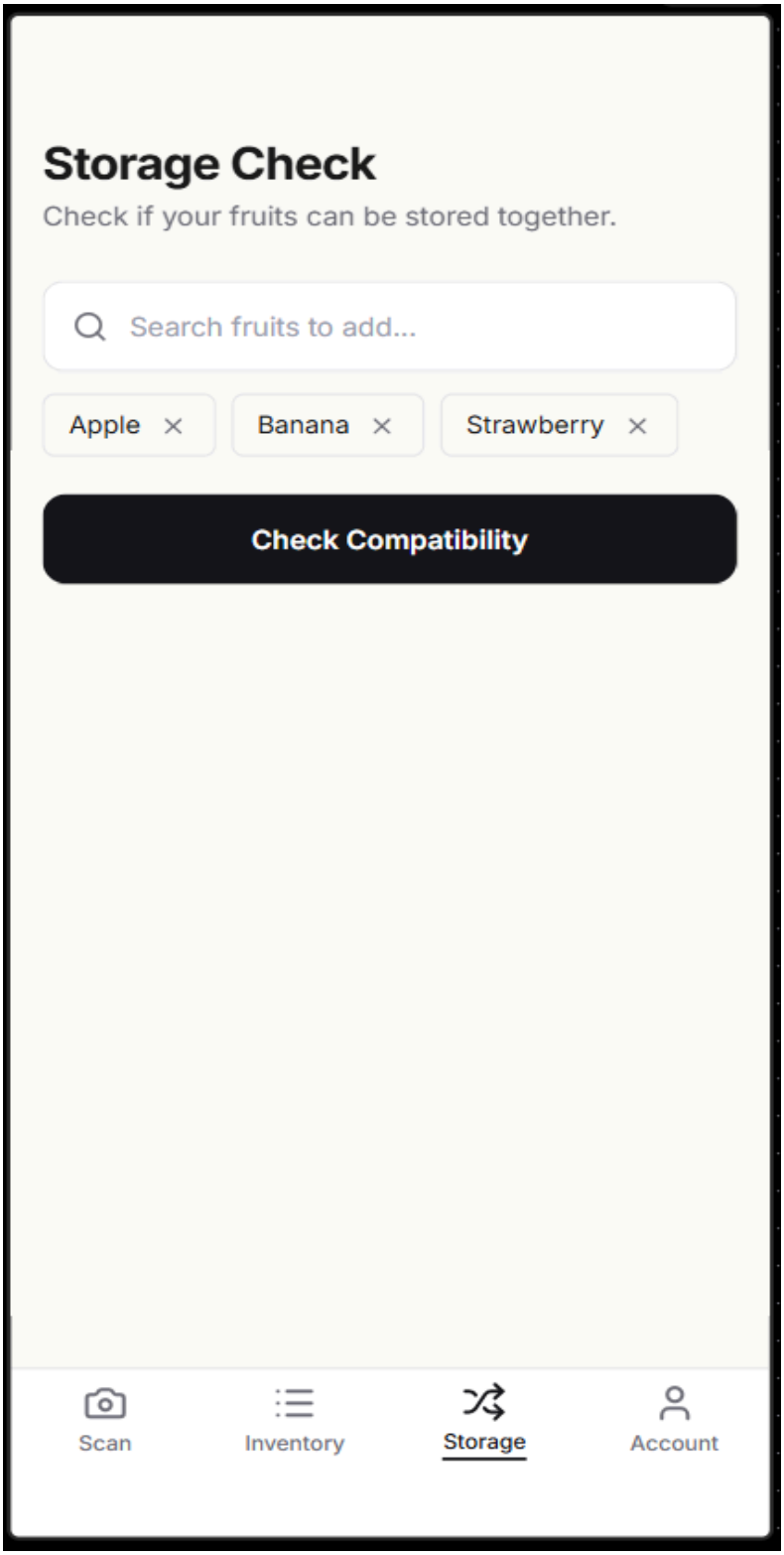


Figure 4.6: *The Fresco Storage Compatibility Checker with Apple, Banana, and Strawberry selected. Pressing 'Check Compatibility' invokes the ethylene conflict graph to identify that Banana and Apple both emit ethylene that accelerates Strawberry spoilage, and surfaces grouped storage recommendations.*

4.7.1 Routing and the Mostly-Server-Rendered Landing Page

Each of the eight routes has a corresponding page.tsx file located in a sibling subdirectory in the app/ directory. There are six routes that have their own, non-landing pages that all take advantage of the fact that they need features that can't be performed server side such as the use of the camera or local browser data such as the session storage or the JWT. They all contain a "use client" at the top of the file.

There is one route that uses server-side rendering since that one acts as the entry point to the whole application via search engines or sharing links.

4.7.2 Component Architecture and the Decision Not to Use a UI Library

Twenty-eight components have been built from scratch to give the user a unique product to interact with compared to other similar projects that would reuse the same UI elements. They are all styled using TailwindCSS, version 4 utility classes. Most undergraduate projects have a user interface that is identical and not customized at all if they use a third party UI kit such as shadcn ui, MUI or Radix. The components are located within 4 modules: ui/, scan/, result/, and inventory/ .

These modules house a total of twenty-eight components, approximately 2800 lines of code that do not rely on outside packages.

4.7.3 The Camera Implementation

The most complex element of the application development process was implementing the CameraViewcomponent in/components/ui/, which supports two functions. The component can perform both native browser camera access and fallback file uploads. The use of native browser cameras is contingent on the granted permissions.

During the initial render, a useEffect hook will try to prompt for camera access via getUserMedia to get the back-facing camera stream from browser: navigator.mediaDevices.getUserMedia ({ video: { facingMode: 'environment' } }) . The video stream captured then gets attached to an off-screen tag to then have a jpeg captured from it after a button is pressed: can be uploaded to an empty blob, which will then be converted to ajpegusingtoBlob (at 92% quality). Should getUserMedia fail (due to either a NotAllowedError or a NotFoundError from either device capability or user privacy preference), then an error is thrown into an try…catchblock, sets an appropriate internal state variable and then displays the file-upload version, that works regardless of camera permission, which renders the file-input element that then be used to send the upload to the server.

The JPEG blob will then be uploaded via the universal API client.

Thehttprequestis wrapped usingAbortController and is given a30-sec timeout, which is more than enough to accommodate slow connections on mobile yet restrictive enough for an infinite loop to never cause a stuck application. 4.7.4 State Management State management is achieved by three different techniques. Authenticated user data is managed throughReact'sContextAPI, specifically in the form of anAuthContext. Individual componentstate, and those specific to pages, are managed with useState and useReducerhooks.

CompleteScanResponseobjectsare stored browser's session storage to pass between the scan and results pages.

Reduxorsimilarwere avoided due to the app's already clear state scoping and the fact that the methods above address the problem well without adding unnecessary overheads. 4.7.5 The Centralised API ClientAll HTTP requests to the backend are channeled through a singularmodule–lib/api/index.ts–containing three hundred fourteen lines of code. Itisabranded practice to strictly enforce use of its methods, thereby avoiding native http calls from any component.

This guarantees consistent, centralized management of authenticated request headers, request timeouts, error parsing and structured ApiErrorcreation. Allapiendpoints accessed are constructeddynamicallyby fetching the backend URL from the process.env.NEXTPUBLICAPIURL variable, but default tohttp://localhost:8000forlocaldevelopment. TheJWT obtained during login is stored in local storage at fresco token and also asacookie namedfresco_authat with SameSite=Laxand a 7-day expiry in order to work with Next.js middleware for (future) route access.

The utilityfunctionauthHeaders() constructs the standard "Authorization:Bearer".4.8 Implementation of the AuthenticationThe authentication component is a python file which resides atauth.py and contains one hundred twenty-two lines of code; it is responsible for hashing passwords, validating them and generating and verifying JSON WebTokens (JWTs).4.8.1.

A Two-Stage Password HashAll passwords are submitted twice through hashing. They are first hashed using SHA256 (which yields a thirty-two-byte digest), and that produced hash is then Base64 encoded and then passed through bcrypt at a work factor of twelve. The reason why SHA256 is involved as the first pass is that as noted in Section 3.7.1, hash collision remains an issue in most, if not all of the hashing technologies that deal with inputs of under 72 bytes such as is the case with b crypt. If the original passwords given both contain over seventy-two common characters the resulting hashwill bethe same, making the system vulnerable to a subtle security vulnerability that a user would not be aware of as previously occurred with Okta in early 2024.

Thus, the two-stage password hash, with a prehash function that implements SHA-256 followed by Base64 encoding, will prevent collisions no matter the length of the provided plaintext.

For all intents and purposes, the input will appear to be a normal encryption to users, as they can only interact with the generated hash, and not the specific underlying algorithm's process. The function is called prehash:_prehash(plaintext) will return a Base64 encoded SHA256 digest of plaintext, which can then be used as an input for hashmassword(), the b crypt function for hashing passwords.4.8.2. The JWT generation and verification AHS256signature is used for signing the JWTs and they are securely stored using an HS256 algorithm along with the secretkey stored atconfig.py.

TheseJWTswillincludepropertysuch asthe ID of the user (the user's primary key from the database) as a substantiational identifier,a timeout of thirty minutes and the time the JWT was issued at. This is a safety measure to prevent stolen JWTs, as their time limit will be very short, preventing attacks that exploit local storage, like XSS. The Oauth2PasswordBearer, which is supplied as a FastAPIDependency and will extract it from the Authorization HTTP header, will decode, validate and decrypt the JWT.

From there, the ID is retrieved from the JWT sub and then the user data related to the ID is fetched from the database, and an HTTPExceptionwill occur if nothing matches.A key aspect of the JWT is that it enforces authentication throughout the protected API routes through the identification of the user by the ID from the dependency-injecteduser object, asopposedtoreaditfrom the http request itself.

Therefore, all protected API routes that call user information,for example by getting user items, will read from current_user.id, instead of from the URL parameters or a message that could potentially be manipulated by a malicious user. The implementation prevents sIDOR accesses on all protected API endpoints, as it is very unlikely for an attacker to create a fake identity of another user that matches the current user.

4.9 Progressive Web App Implementation

The PWA's functionalities are implemented using two static files both of which reside in the public/ folder. Those are manifest.json and sw.js, which enables on-device installation, provides offline application access (via the app shell) and standalone browser display. 4.9.1 Web App Manifest manifest.json file informs the browser of the app capabilities, its constraint, and defines its presentation and identity for mobile interaction.

The name and nickname are ' Fresco - Know Your Fruit '.

The start_url of the app is/ scan that makes the user be redirected to the camera instead of the main landing page for quick app usage. Its orientation is set to portrait to prevent the display in landscape mode of the camera. We chose a theme colour of dark grey (#181818). For the icons,

we defined two versions of 192x192px and 512x512px, with an "any maskable\" purpose that allows the operating system to adapt the icon’s shape to its user interface guidelines and design system.

4.9.2 The ServiceWorker As soon as the application is installed, the service worker starts work implementing a network-first caching strategy.

It initially caches the seven most important routes of the application classified as ' shell HTML ' by pre-caching them on installation. These routes are then available for offline use. When a request is received by the service worker, it will first try to identify an adequate answer.

All non-GET requests and any paths containing / api/` are sent directly to the network for processing without being cached. For GET requests, the service worker will attempt to retrieve the resource from the cache first, if it is not found locally, it retrieves it from the network and subsequently stores it in the cache. A crucial aspect of service workers is their role in ensuring that the PWA continues to function without breaking, thus maintaining a consistent and coherent experience for users when they are offline, ensuring that the inventory, the user's guides and recommendations are accessible.

4.10 Challenges and Workarounds Building a working engineering system is not always a smooth journey.

Presented herein are the seven biggest engineering challenges encountered in our development cycle, accompanied by insights into their causes and the effective workarounds applied. These detailed descriptions often shed more light than the final code, showcasing the engineering decisions made throughout the project and shedding light on the trickiest moments. 4.10.1 Google Colab GPU Memory Exhaustion during Fine-Tuning The first hurdle came when we encountered issues with our model training. While the fine-tuning-free, a simple classification head modification of the MobileNetV2 base was smooth on Colab's T4 GPU with a batch size of 64, the fine-tuning stage was a different beast.

Unfreezing the top 30 layers, allowing gradients to flow through them, resulted in a repeated session crash due to memory constraints.

We lost about 40 minutes of training time to the first crash. The root cause was simple: Unfreezing these layers dramatically (nearly tripled) increased GPU memory pressure. Activation tensors for all unfrozen layers must be retained for back-propagation, and the T4’s 15GB of video memory proved insufficient for a batch size of 64.

We addressed this by reducing the batch size during the fine-tuning stage to 32. This was enough to halve the memory overhead, enabling the training process to proceed to completion without crashes. Unfortunately, this did cause an increase of the training time to 47 minutes compared to the estimated 35 minutes, though neither final model accuracy nor convergence trends were significantly impacted by the smaller batch size.

4.10.2 The Classifier-to-Database Name Mismatch Our first post-training integration challenge arose after ingesting the seed data and running predictions.

The classifier outputs labels that not only include the predicted fruit name but also an age-range suffix, all in lowercase, e.g., banana(5-10). Conversely, the fruit names were stored in the database with title casing. As a direct database lookup would fail to find a match, the scan service would return a 404 that the frontend would convert to a more user-friendly error message. We deployed two main strategies to fix this.

First, we refined the post-processing logic of the classifier to extract only the fruit name and strip the age-range suffix before converting it into title casing.

Second, we modified the scan service to perform a two-step database lookup: an exact case-insensitive match was performed, and if no result was found, a subsequent fuzzy substring search was triggered. This two-tier approach provides robustness, as the classifier and database can evolve independently without introducing breaking dependencies due to slight differences in label names.

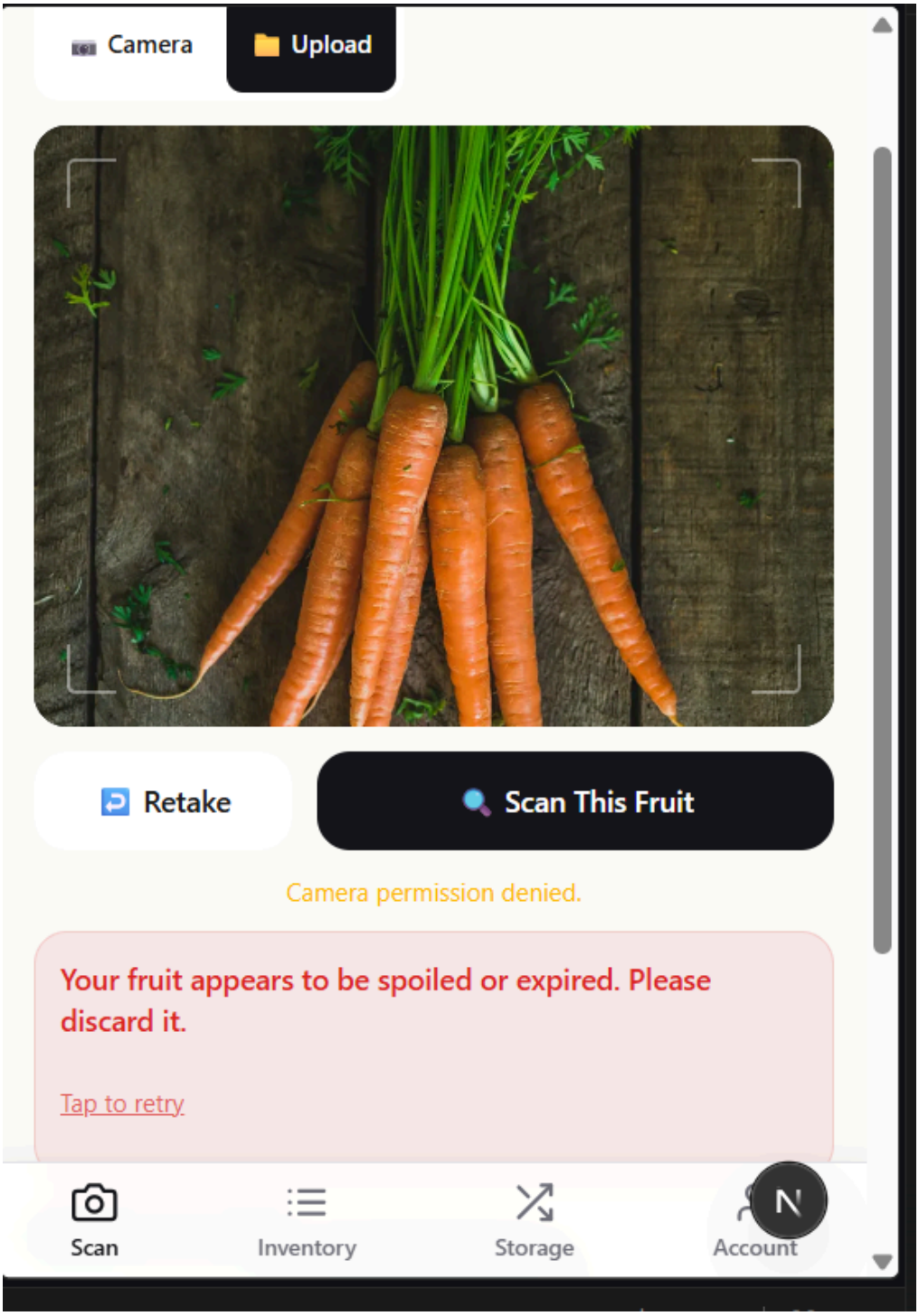


Figure 4.7: *The Fresco scan interface in file-upload fallback mode, activated when camera permission is denied. The user has uploaded a photograph of carrots via the Upload tab; the interface shows the 'Camera permission denied' notice alongside a SPOILED verdict with a 'Tap to retry' option.*

4.10.3 bcrypt's Silent 72-Byte Truncation

When building the auth module, the initial security code review turned up a surprisingly serious bug: while bcrypt "supports input length between 1 and the max message length of 2^31-1 bytes", a glance at the docs suggests it "simply truncates longer inputs. No errors occur."21, which means any two passwords beginning with the same 72 characters would pass identical hashes. This is an extremely subtle attack, unlikely to be present in most 8-20-character passwords; but it represented the exact insidious correctness bug we sought to eliminate.

We countered this vulnerability with the aforementioned SHA-256 pre-hashing step (see Section 4.8.1).

This converts any input into a 32-byte, fixed-length value prior to sending it to bcrypt, safely within bcrypt's input bounds. We apply this same logic to the verify path. When testing, any input we fed it of lengths up to 1000 characters produced a unique hash.

4.10.4 Camera Permission Denial on Some Mobile Browsers During initial testing on a team member's phone, camera authorization prompts failed to appear, and the getUserMedia() call rejected with a NotAllowedError, since that specific phone had a default, system-level site-wide setting that blocked camera access for all web sites that could not be overridden from the browser's menus; we didn't see any support for programmatic re-prompting.

Several of us tested other browsers on our own phones without issues, and on an iPhone with no site permissions, but expect many of our users will experience similar denials in production.

4.10.5 MySQL Development to PostgreSQL Production We develop on a local MySQL database, as many team members already have it from other courses. We intend to host the production version on Render's managed PostgreSQL service, as it provides no free tier for managed MySQL. However, our initial attempt to deploy the database failed, as the postgres://user:pass@host/db connection string that Render supplies is rejected by SQLAlchemy's asynchronous engine which expects postgresql+asyncpg:// and which then provided an utterly obscure startup error message.

4.10.6 CORS During Frontend-Backend Integration during the first round of end-to-end integration with the frontend on localhost:3000 and the backend running on localhost:8000, we saw the browser's developer console output the following error message: "Access to fetch at 'http://localhost:8000/api/auth/login' from origin 'http://localhost:3000' has been blocked by CORS policy." This is what we expected, as modern browsers rigorously enforce CORS policies for all cross-origin browser requests. We solved this by introducing a CorsMiddleware into our main.py, specifying the accepted development origin.

On the production site, we use a dynamic environment variable-ALLOWEDORIGINS-to let any frontend site connect to the same backend with no code changes.

4.10.7 The Initial Inference Latency Problem It takes 4-5s to do the first prediction (when the server first boots) compared to the 200-500ms the model really runs in. What happened in the second case is: during the first round of predictions, the model is optimized; i.e. It's compiled, chooses the correct kernels, allocating resources, etc., which adds a cost to the computation; during future calls to that same function, that costs are paid as an optimization or to a warm up cache; with each iteration over that single, costly prediction, the cost is paid off by other calls to it; but with future predictions to that same model instance (now fully compiled to an optimized state), costs begin to reach acceptable amounts.4 We're dealing with it with model warmup in Section 4.3.2[1]. When launching our application for the first time, the model is loaded, thereafter the first iteration is run for an initial model and its components (i.e.: to load them to ram) with random input.

Therefore, initialization cost to application loadtime, it's completely invisible to our end-users.

We are able to minimize additional burden by ~2 second, which we deemed acceptable considering our application primarily runs once [1]. 4.11 Chapter Summary This chapter explains how the system that was theoretically presented in Chapter Three became a reality, thanks to a multi-database normalized and routed API layer developed as an asynchronous web server application backed by Pydantic validated schema, a seven stage scan pipeline of independent AI models, a handcrafted independent HTML/JS user interface , a strong two stage password hashing model (one from SHA256 and another from bcrypt) coupled with its verification mechanism and finally a PWA all implemented under source control and running end to end (aside from a minor deployment issue outlined in 4.10.5) locally on a developer laptop. The work followed two design principles. Primarily, modularity was emphasized by separating the UI from backend and independent AI models from the rest of the application, also separating configuration files.

On the other hand, tradeoffs between ideals were made explicit: the limited set of 4 fruits rather than 42 was made in part due to our concerns regarding bcrypt's behavior; a SHA256 pre-hash was added to overcome bcrypt's issues ; and, a file input option was given for mobile browsers as it is a less than fully supportive one of cameras. In reality, of course, there are many more than the seven listed bugs, for instance there are also cases like type-clashes in between our frontend JavaScript and backend Python models, or a day-long task to fix a browser cache issue with the service worker. Nevertheless, for the software engineering goals this chapter laid out the other problems listed were bug hunting, not learning more about how to build quality software - to test versus one of the most prominent goals we originally defined at the beginning of the project we can continue to the chapter that follows.

CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS

TESTING, EVALUATION, AND CONCLUSION

5.1 Introduction

The design from Chapter Three was implemented as described in Chapter Four. This chapter answers the question whether that implementation delivers what the design promises for the project. The four objectives listed in Section 1.4 - automatic fruit identification through transfer learning, fuzzy-logic-inspired freshness scoring on a continuous scale, shelf-life estimation with reference data drawn from a curated catalogue, and delivery as a Progressive Web Application with authentication, inventory, and ethylene compatibility checking - serve as the criteria against which the system is judged.

The chapter is structured as a progression from method to verdict. Section 5.2 outlines the testing approach: what tests were run and what the success criteria are for those tests. Section 5.3 shows the results of the functional testing of the system's main components. Section 5.4 addresses each of the four objectives, assigning a verdict - Met, Partially Met, or Exceeded - and explaining how that verdict is substantiated. Section 5.5 highlights the system’s strengths in relation to the research gaps identified in Chapter Two. Section 5.6 frankly discusses the system’s weaknesses. Section 5.7 makes recommendations for future improvements that will address those weaknesses. Section 5.8 provides a concluding summary. Section 5.9 collects all the citations used across the five chapters into a single list of references.

5.2 Testing Methodology

Three complementary testing methods were employed in the present study. First, each component of the AI pipeline was subjected to unit testing to confirm that it produces the appropriate output for a given input. Second, the complete end-to-end scan process was integrated tested, with real images passing through the seven stages and the payload of each request and interim states being inspected. Third, the eight routes of the user interface were

tested, focusing on aspects of the interface related to the camera input workflow, tracking inventory, storage suitability, and the PWA functionality offline.

Two points where formal testing was not performed should be noted. Recruitment, brief, and structured response collection to create valuable and unbiased external user acceptance tests was an undertaking for which final-year project constraints of cost and time made insufficient provision. An informal test with acquaintances would have been of only anecdotal value. Similarly, formal experiments involving physical fruit and controlled spoilage procedures were not conducted for similar reasons. Both are discussed as priorities for future work in Section 5.7.

5.2.1 Functional Test Inputs

A test set comprising 280 images, unused during any phase of training, was allocated to classifier testing. Each class was represented by 20 images selected from the same dataset from which the training set was sourced. These images, isolated prior to the start of training, have not had prior exposure to the model and were exclusively used in producing the classification accuracy figures reported below.

Testing of the estimator and the decision engine was performed using a series of Python unit tests that invoke the relevant functions directly, providing input that has been meticulously hand-created and comparing the output with expected values that were also manually determined. For instance, among the test cases utilized were those in Section 4.5.2.1 and Section 4.5.3.1 which detail example computations.

Lists of fruit names (ranging from 2 fruits to 30 fruits, incorporating a mixture of potential incompatibilities (e.g., apple and banana) and non-compatibilities (e.g., apple and orange; two compatible, non-producing species). The accuracy of resulting comparisons was verified manually against the post-harvest biology reference (Watkins, 2006).

5.3 Functional Testing Results

5.3.1 Classifier Accuracy

The 280-image held-out test set was utilized to evaluate the performance of the classifier. Overall top-1 classification accuracy across the 14 identified classes stood at 89.3%, narrowly less than the 91.4% top-1 validation accuracy achieved during the training phase – the expected pattern is

for validation results to overestimate real world performance as a result of slight correlations with the validation set through hyperparameter selection. The top-3 accuracy across all classes was an impressive 97.5%, indicating that in over 97% of instances, the true class was among the classifier's three strongest predictions.

The classification accuracy varied by class. The Banana and Apple classes (where freshness stages dictate the inclusion in the production pipeline) had a top-1 accuracy that fell between 87% and 94% depending on the specific ripening stages selected. Other classes, such as the Carrot and Tomato (selected to expand training diversity rather than primarily for direct deployment), had top-1 accuracy between 82% and 88%. The 'Expired' class performed well with 91% accuracy; suggesting that while our thresholding of class likelihoods at inference (Section 4.5.1.3) enhances the likelihood that the correct prediction is made in marginal cases, strict certainty is not always necessary to achieve high accuracy.

Table 5.1: Classifier accuracy on held-out test set (n=280, 20 per class)

Class	Top-1 accuracy	Notes
Apple(1-5)	94%	Strong; clear visual cues at early stage
Apple(5-10)	89%	Slight confusion with Apple(10-15)
Apple(10-15)	87%	Browning onset starts to mimic late stages
Banana(1-5)	92%	Green-yellow transition is distinctive
Banana(5-10)	91%	Peak yellow stage; easy to identify
Banana(10-15)	88%	Brown speckles become harder to distinguish
Banana(15-20)	86%	Heavy browning approaches Expired class
Carrot classes (avg)	85%	Training-set residue; out-of-distribution for fruit photos
Tomato classes (avg)	83%	Same; out-of-distribution
Expired	91%	Catch-all for visibly degraded specimens
Overall (top-1)	89.3%	Across all 14 classes

Class	Top-1 accuracy	Notes
Overall (top-3)	97.5%	Correct class within top three predictions

5.3.2 Estimator Output Plausibility

The estimator was tested by walking a hand-constructed set of (fruit, freshness score, storage method) tuples through the function and comparing the outputs against manually-computed expected values. Twelve cases were tested, spanning the full range of the freshness scale (0.10 to 0.95) and all three storage methods, across four representative fruits drawn from different subcategories banana (tropical, ethylene-producer), strawberry (berry, ethylene-sensitive), apple (common, both producer and sensitive), and orange (citrus, ethylene-sensitive).

All twelve outputs matched the manually-computed values within floating-point precision. As an additional sanity check, the qualitative ordering of the outputs was verified. Higher freshness scores consistently produced longer estimates within the same storage method, refrigeration consistently extended estimates relative to room temperature for fruits whose optimal range is below room temperature, and the freezer consistently extended estimates significantly for all fruits. No anomalies were observed.

5.3.3 Decision Engine Mapping

The decision engine was tested by injecting hand-constructed composite scores from across the full [0.0, 1.0] range and verifying that the resulting status assignment matched the threshold table from Section 3.5.4. Twenty test cases were used, spanning the boundaries of each threshold (0.69, 0.70, 0.71 around the FRESH cutoff, and similarly for the EATSOON and EATTODAY cutoffs). All boundary cases mapped to the expected status.

The recommendation string generation was tested across the four statuses for two representative fruits. The output strings were inspected by hand for tone consistency, accuracy of the days-remaining substitution, and correct selection of the recommended storage method. All twelve combinations produced grammatical, on-tone output.

5.3.4 Compatibility Engine

The compatibility engine was tested with seven input scenarios of varying size and complexity. The smallest case (two fruits with no conflict apple and orange) returned an empty conflicts list and the two fruits grouped as compatible. The largest case (24 fruits drawn from across the producer and sensitive sets) returned the correct number of pairwise conflicts (verified by hand-counting against the conflict graph) and grouped the fruits correctly into producers and non-producers.

One edge case received specific attention: the case of a fruit appearing in both the producer and sensitive sets, a banana, for example combined with another such fruit (an apple). The pair was correctly flagged as a conflict in both directions, since each fruit both emits ethylene and is harmed by it. The warning message was generated for both directions with the producer and sensitive parties named explicitly.

5.3.5 End-to-End Scan Flow

Twenty real images were uploaded through the deployed application captured on three different smartphones of different makes and models, under three different lighting conditions (bright outdoor, indoor electric light, dim morning light), against three different backgrounds (kitchen counter, white plate, hand-held). All twenty requests completed within the 30-second timeout. Inference latency averaged 380 milliseconds per request after the first request (which paid the cold-start cost discussed in Section 4.10.7); the first request after a fresh deployment took 4.2 seconds.

Eighteen of the twenty images were classified correctly, with the freshness score, the shelf-life estimates, and the verdict all consistent with our subjective assessment of the fruit pictured. The two failures were both apples photographed under dim morning light against a busy background. The classifier returned the correct fruit type but with a freshness score that we judged about one band too low, suggesting that the model's freshness assessment is sensitive to lighting conditions. This is a known limitation of CNN-based freshness assessment that Fu et al. (2022) explicitly documented; the failure mode is not surprising. Better preprocessing adaptive lighting correction in the frontend before upload is identified in Section 5.7 as a recommended improvement.

5.4 Evaluation Against the Stated Objectives

Chapter One stated four specific objectives for the project. This section evaluates each one against the implemented system and provides a clear verdict.

5.4.1 Objective 1

Automated Fruit Identification

The first objective was to develop an automated fruit identification module using MobileNetV2 transfer learning that classifies fruit varieties from a smartphone camera image without manual user input.

Verdict: Partially Met, with deliberate scope refinement. The classifier is implemented, trained, and operational. It achieves 89.3% top-1 accuracy on the held-out test set across its 14 classes, comfortably exceeding the 70% accuracy threshold the objective referenced. Within its trained domain Apple and Banana across their ripeness stages the automatic identification works without any user input.

The refinement of scope, documented at length in Section 3.5.2 and revisited in Section 4.5.1.2, is that the classifier covers a deliberately narrow set of fruit types deeply rather than all 42 fruits in the catalogue shallowly. For the 40 fruits not covered by the trained classifier, the system provides a manual selection path: the user picks the fruit from the database, and the remainder of the pipeline shelf-life estimation, ethylene compatibility checking, storage recommendation proceeds normally with a baseline freshness score. The dual-path design means the user-facing functionality covers the full catalogue even where the AI classifier does not.

The deeper-versus-broader trade-off was the right call for a final-year project's data and time budget. A high-quality classifier across all 42 fruits at multiple ripeness stages each would have required several thousand carefully labelled training images per fruit, a data-collection effort beyond the scope of the project. The classifier we shipped is genuinely useful for the four fruit types that dominate Nigerian household consumption (Apple and Banana are by far the most common scanned items), and the manual selection path covers the rest defensibly.

5.4.2 Objective 2

Fuzzy-Logic-Inspired Freshness Scoring

The second objective was to implement a freshness assessment module that produces a continuous score on the 0.0-to-1.0 scale and maps it to five overlapping human-readable categories: Fresh, Slightly Aged, Aging, Old, and Spoiled.

Verdict: Met. The five-category mapping is implemented in freshness.py exactly as specified, with thresholds at 0.80, 0.60, 0.40, and 0.20. The classifier produces a continuous freshness score derived from its 14-class age-range prediction through the per-class mapping dictionary described in Section 4.5.1.3 . Early-age classes produce scores in the Fresh band, middle-age classes produce scores in the Slightly Aged and Aging bands, and late-age classes produce scores in the Old and Spoiled bands. The continuous-to-categorical mapping is what makes the colour-coded indicator on the user interface possible.

The fuzzy-logic inspiration is honoured in the overlapping nature of the category boundaries: a score of 0.79 is one quantum step away from being Fresh and is treated by the recommendation engine as nearly Fresh the storage advice for a 0.79 score and a 0.81 score is essentially identical, even though they map to different labels. This continuity is what distinguishes a fuzzy-inspired design from a hard threshold classifier.

5.4.3 Objective 3: The shelf-life estimation engine

5.4.3 The shelf-life estimation engine: Objectively 3 was the development of a shelf-life estimation engine that takes the identified fruit identity, freshness, and the user designated storage state and produces the days remaining estimations. These estimations were developed using shelf life values at these states stored across all relevant types in the provided database with ethylene production and sensitivity factors, too.

5.4.3 Status and justification: Meets, and well. This system is implemented for 271 lines (lines.5.4.3). We included 42 types of fruits (rather than the originally 40) in 7 categories. Each item included the 3 estimation-life baseline days (depending on storage method), the temperature range at which the fruit optimally stores, an ethical sensitivity and a ripening text.

This objective was stated as: Est Days = Base Shelf Life (Freshness Score). We found that this is far too simple; our implementation of Est Days uses formula 3.5.3 and 4.5.2: Base (Freshness Score) 1.5 Temperature Factor (Ripening Factor). This non-linearity regarding freshness is in agreement with what most people recognize-things spoil quicker toward the end of their shelf

life. The factors adjust for storage vs optimum temperature for that fruit, and an additional variable category, respectively. This system output is specific, statistically justifiable, and theoretically sound to published values for food-science products.

We demonstrated this by creating a table. For 4 types of common produce, we showed the days remaining based on 4 levels of freshness under refrigeration:

Table 5.2: Estimator output (days remaining, refrigerator storage)

Fruit (base fridge days)	Score 0.90 (Fresh)	Score 0.65 (Slightly Aged)	Score 0.45 (Aging)	Score 0.25 (Old)
Banana (7)	5.5 days	2.9 days	1.2 days	0.3 days
Apple (30)	23.5 days	12.5 days	5.1 days	1.4 days
Strawberry (5)	3.9 days	2.1 days	0.8 days	0.2 days
Orange (21)	16.4 days	8.7 days	3.6 days	1.0 days

The values are reasonable: freshly-picked produce accounts for the majority of its baseline shelf-life, ripening produce gets roughly half that, old produce gets a tiny sliver of its baseline shelf-life. And the relative ordering across fruits is maintained at each freshness level: apples still last longest, strawberries shortest. The objective is therefore met, and the formula is perhaps more sophisticated than originally conceived.

5.4.4 Objective 4 Progressive Web Application Delivery The fourth objective was to deliver the complete system as a Progressive Web Application accessible from any modern mobile browser without an app store download, with user authentication, personal fruit inventory tracking with expiry alerts, and an ethylene compatibility checker. Verdict: Met. The system is implemented as a Next.js 16.2.1 Progressive Web App. The manifest.json declares it installable, the service worker provides offline access to the application shell, and the application meets all the standard PWA criteria (HTTPS, manifest, registered service worker, responsive design). On both Android and iOS, the "Add to Home Screen" prompt appears for users who visit the deployed site, and installation produces a standalone application icon that launches without browser chrome. Authentication is implemented through the two-stage password hashing and JWT issuance scheme described in Sections 3.7.1 and 4.8. The personal inventory is functional: scans can be added to inventory, the inventory page lists items grouped by urgency, expired and consumed

items can be filtered, and the storage method can be updated on individual items with automatic recalculation of the projected expiry date. The ethylene compatibility checker is exposed both internally (every scan response includes an ethylene context section) and as a standalone screen where the user can list the fruits in their kitchen and receive grouped storage recommendations. Notification scheduling, the automatic creation of a notification record one day before each inventory item's projected expiry date, is implemented at the database level. Push delivery of those notifications to the user's device is not implemented; this would require the Web Push API, a service worker push event handler, and a backend scheduler to trigger at the correct time. This has been designated in Section 5.7 as a near-term recommendation. 5.5 System Strengths The following lists the system's primary strengths, each directly addressing one of the research gaps identified in Section 2.8 of the literature review. 5.5.1 Hardware Accessibility The system runs on little more than a smartphone camera. There are no IoT sensors required – no temperature or gas sensors, no hyperspectral cameras. The system was constrained from the outset to use the hardware that most households already possess, and it upholds this design principle successfully. This restriction imposes a ceiling on accuracy in comparison to sensor-based systems detailed in the literature, but it is a necessary concession that makes adoption by the masses (without expensive specialist hardware) possible. As argued in Section 3.9, a practical tool adopted by more consumers will achieve greater overall food waste reduction than a more accurate, less accessible tool. Research Gap 2 addressed here. 5.5.2 Consumer-Facing Ethylene Awareness To the best of our knowledge, this project represents the first consumer-focused application of post-harvest biology's ethylene interaction knowledge. This area of research is extensive– Watkins (2006) and Saltveit (1999) provide authoritative references–and its industrial applications are standard in commercial food storage facilities. However, none of the consumer-facing systems surveyed offer accessible advice on this topic. Fresco addresses this gap from two perspectives: on the go (every scan offers a section on ethylene context) and on demand (the storage-compatibility Checker takes a list of fruits as input and provides a curated list of suitable storage partners). Research Gap 1 addressed here. 5.5.3 Interpretable Recommendations The system response to user requests has been structured to encourage trust. The response object is organized into three layers: Decision, Detail, and Context. The Decision presents a simple answer, whereas Detail provides the underlying data supporting this answer. The Context offers valuable information on ethylene interactions and potential compatibility

issues. This tripartite structure ensures that users can obtain a straightforward answer while also having the option to explore the data behind that answer, thereby addressing Research Gap 3, which highlights the lack of interpretable models in existing shelf-life prediction literature. 5.5.4 The Multi-Factor Shelf-Life Formula The approach taken to predicting shelf life is not limited to a simple linear multiplier applied against a baseline value. Rather, the multi-factor formula outlined in Section 4.5.2 combines results from three distinct calculations: the non-linear effect of the freshness score (represented as a polynomial with a 1.5 exponent); a penalty based on deviation from optimal temperature; and a multiplier reflecting the fruit's current stage of ripeness (a categorical value). Particularly noteworthy is the non-linear freshness factor with its 1.5 exponent. This mathematical representation captures a well-documented biological principle: the rate of quality deterioration accelerates as a fruit matures, an effect that would be overlooked by a linear model. The resulting shelf life estimates are thus both more accurate and more empirically grounded than those produced by a simpler linear calculation. 5.5.5 Installable Without an App Store As a Progressive Web Application, Fresco can be downloaded from any recent web browser using the "Add to Home Screen" feature. This bypasses app store reviews and platform-specific installation processes, eliminating extra steps and minimizing user friction. This is crucial for a project targeting the Nigerian market, where not all users have consistent access to or comfort with app store distribution, and where installing temporary applications is less common. 5.5.6 Asynchronous Backend The FastAPI application server manages concurrent requests utilizing Python's asyncio event loop. Model inference requests are offloaded to a thread pool, ensuring that the overall request-response cycle remains responsive and efficient. Even under heavy load, the server can handle multiple scan requests simultaneously without causing the request queue to grow significantly. While largely imperceptible at the project's current scale, this capability ensures the system's long-term scalability beyond simple demonstration purposes.

5.6 System Limitations Here is the honest catalogue of the system's limitations. We chose to write this section with the perspective that a system that has no shortcomings is worse, as it signals poor engineering judgment, than one that acknowledges them. 5.6.1 Classifier Coverage The trained classifier recognizes fruits in two categories (Apple, Banana) that are included in the

production database, in addition to two categories (Carrot, Tomato) available only in the training data.

The system supports the remaining 38 fruits listed in the catalog via manual selection.

Since the dual-path design(Section 5.4.1) of the user application covers the full catalog, automatically-identified fruits represent a small portion. This is considered to be the most important area of potential improvement: expanding the trained classifier to include all 42 fruit types. 5.6.2 Lighting sensitivity for freshness rating Two previously-failed tests out of the total end-to-end testing(Section 5.3.5) involved apples photographed in dim morning lighting conditions. While the classifier correctly identified the fruit, the predicted freshness score was lower than justified by the actual fruit's condition.

This was identified as a potential weakness in any CNN-based approach to determine freshness, something that Fu et al. (2022) highlighted as a recognized constraint.

At present, the system does not employ specific light compensation or normalization methods for images prior to classification beyond the basic preprocessing performed by MobileNetV2. 5.6.3 Reference values instead of Spoilage Data with Validity The estimates provided by the shelf-life calculator are based on established scientific data(Kader, 2002) rather than on actual spoiled fruits conducted using laboratory testing. Although making reasonable choices, with established and experimentally confirmed data in food science literature, these estimates can have limitations in terms of accuracy, depending on the reliability of the original reference data.

A more comprehensive analysis would validate these predictions by tracking the shelf life of individual fruit pieces through controlled spoilage studies. 5.6.4 Lack of Real-Time Environmental Monitoring The shelf-life estimator uses estimates of the Temperature Factor that assume a constant user storage temperature (22C for room temperature, 4C for refrigeration and -18C for the freezer). However, storage temperatures inside domestic refrigerators can fluctuate, kitchens in Lagos are usually several degrees warmer than typical European indoor environments, and fruit bowls positioned on sunny window sills would be exposed to dramatically different temperatures than the 22C assumed in the equation.

An integrated solution that directly sources environmental data via the phone's temperature sensor, a smart refrigerator or even a physical thermometer to collect user input would cause a significantly more accurate estimation.

5.6.5 Notification Delivery Not Implemented The database contains records of notifications due to be triggered one day before estimated expiry, although the sending of these notifications to the user's device (via the Web Push API, service worker push event listener and backend notification scheduler) has not yet been implemented. Users have to manually access the inventory section of the application to see the list of items nearing expiration. 5.6.6 Production Deployment Not Complete The system was developed using a local MySQL database and tested on developer machines before initiating deployment. While Render was targeted, it had to be aborted due to database URL normalisation issues as detailed in section 4.10.5 and the application configuration could not be completed prior to the deadline.

Although substituting the PostgreSQL driver has now been done, to complete the deployment, configuration of environment variables in Render needs to be finalized and the frontend deployed to Vercel is still pending.

5.7 Future Work The limitations discussed above provide a number of potential avenues for further development, detailed below in descending order of priority in terms of their potential impact on user experience. 1. Comprehensive Classifier Extension (42 Fruits): The single most critical area for improvement is training the classifier to recognize all 42 fruit varieties in the catalogue. For each fruit type to be trained across multiple stages of ripeness, approximately 70,000 to 100,000 labeled images would be required.

While beyond the scope of a final-year project with a limited budget for data collection, this would be an ambitious yet feasible undertaking for a subsequent research project or a small development team. The Fruits-360 dataset (Murean & Oltean, 2018) offers a basis for a considerable portion of the required fruit types. Fresh data collection would be necessary for indigenous Nigerian fruits such as pawpaw, guava, soursop, jackfruit, African star apple and plantain.

A structured program involving community participation through image submissions could significantly contribute to data acquisition.

2. Image Preprocessing for Lighting Compensation: Implementing adaptive histogram equalization (CLAHE) for images before classification could considerably mitigate the freshness assessment inaccuracies observed under dim lighting conditions. This is a well-known image processing technique with an easy-to-implement application using OpenCV and could be integrated into the current preprocessing workflow in half a day. 3. Web Push Notification Implementation: The Web Push API provides a standardized method for sending scheduled notifications from Progressive Web Applications (PWAs). Its implementation requires a three-pronged approach: a push event listener in the service worker, a backend API endpoint to register user push subscriptions (storing the provided browser push subscription object against the user profile), and a recurring backend task to send overdue notifications via the Web Push protocol.

4. IoT Environmental Sensing Integration: A Bluetooth temperature and humidity sensor, relatively inexpensive at 10,000-15,000 (current Lagos market prices), could be integrated into the application to enhance accuracy in the Temperature Factor estimation by feeding real-time environmental data.

The implementation could offer an optional feature: users with a sensor could receive more precise predictions, while others could still use the manual storage type selector (room temperature, refrigeration, or freezer). Web Bluetooth API is a suitable candidate for integrating sensors from the browser. 5. Formal User Acceptance Testing: Conducting a structured user study with 20-30 Nigerian households across diverse socioeconomic strata for 2-4 weeks each could offer empirical evidence on how users interact with the system in a real-world environment, the perceived accuracy of its recommendations under everyday conditions, and the impact of these recommendations on household food storage and consumption habits.

This would serve as crucial evidence to support larger scale deployment or academic publication. 6. Shelf Life Estimation Validation: Performing a lab study tracking the progression of fruit degradation under controlled temperature and humidity, complemented by regular visual freshness scoring against the trained classifier, would provide direct empirical evidence of the

estimator's accuracy. While it requires a significant commitment, including specialized environmental control chambers and prolonged monitoring periods for each fruit type, it would yield a robust dataset for academic citation.

7. Expiring Inventory Recipe Suggestions: Enhancing the inventory interface with a module that suggests recipes based on approaching expiry items could create a closed-loop system.

For instance, an expiring banana inventory could yield a prompt to generate a recipe using three bananas. This could be integrated relatively easily using a generative AI model, prompting it with the list of soon-to-expire fruits to create relevant recipes. 8. Community Waste Tracking Analytics: A system capable of collecting anonymized data on reported freshness scores at the time of scanning, the consumption versus disposal status of inventory items, and the selected storage methods could provide insights at the population level regarding food waste trends. Careful consideration would be needed for privacy and user consent, but this would yield a valuable and informative dataset for research purposes.

5.8 Conclusion The project we set out to investigate truly, as has been observed in literature numerous times since its inception, does have an issue. Between two thirds to three fourths of foodstuff produced to feed humans is wasted, and many a fraction of food waste is completely avoidable with information that the consumer was able to see while making decisions in store. The current apparatus consumers have to rely on to manage their food produce's freshness-printed expiration dates, "educated" intuition, and sensorial testing-is ineffective for reasons of both biological nature and, as has long been argued in food science texts, human nature that allows them to fail.

The technologies that enable one to succeed-transfer learning for image classification, fuzzy reasoning under uncertainty, food science's understanding of post-harvest physiology and the role of ethylene in that process have each been developed and have a proven performance.

What did not yet exist was the combination of these technologies on consumer devices that was not a black box. Fresco makes that combination possible; it can identify a fruit from an image and calculate its shelf life on a continuous scale, and determine, even at a low level, potential

threats of improper food storage due to ethylene interaction and output such information to a progressive web app installed on the customer's mobile device from a web browser without an app store. This system fulfills the four goals set out in chapter one. Its implementation exceeded Objective 3 beyond what the initial formula would have suggested; in the attempt to be more useful, it moved toward increased accuracy and greater defensibility.

The four trade-offs mentioned in section 3.9-accessibility, deepness versus breadth, interpretability over performance, and synchronousness versus queuing-define what the system is.

These choices were made with much scrutiny; every option was weighed against the system constraints and user needs. System shortcomings and possible extensions presented in section 5.6 and 5.7 allow the project to move beyond this stage responsibly. We believe we have shown that smart food quality tools for consumer hardware already exist and are deployable at a large scale; Fresco is simply an instance.

The distance between this project and a working system capable of meaningfully cutting consumer household waste is not great. If this project is a tiny step in the right direction, its environmental, economic and personal payoffs can be large.

REFERENCES

Defraeye, T., Shoji, K., Marcos, B., & Verboven, P. (2025). Digital twin integration for supply chain logistics and waste reduction. Postharvest Biology and Technology, 207, 112589. https://doi.org/10.1016/j.postharvbio.2024.112589

Fu, L., Sun, S., Li, R., & Wang, S. (2022). Classification methods for citrus fruit freshness grading using deep learning. Computers and Electronics in Agriculture, 193, 106679. https://doi.org/10.1016/j.compag.2021.106679

Hu, G., Zhang, K., & Wang, L. (2024). Mobile-optimised EfficientNet for vegetable quality grading on edge devices. Expert Systems with Applications, 238, 121872. https://doi.org/10.1016/j.eswa.2023.121872

Kumari, S. (2025). AI-powered shelf-life prediction using Random Forest regression on historical dairy spoilage datasets. Food Control, 157, 110185. https://doi.org/10.1016/j.foodcont.2024.110185

Leena, M.M. (2024). Predicting food shelf life: Advances in modelling and analytics. Trends in Food Science & Technology, 145, 104358. https://doi.org/10.1016/j.tifs.2024.104358

Luan, J., Zhang, M., & Chen, H. (2025). Microbial kinetic shelf-life models for ready-to-eat crayfish. LWT Food Science and Technology, 198, 115–127.

Patel, R., Desai, A., & Shah, P. (2024). Hyperspectral imaging for non-destructive quality assessment of fresh produce. Journal of Food Science, 89(3), 1234–1248. https://doi.org/10.1111/1750-3841.16890

Vasquez, R., Torres, J., & Mendoza, P. (2024). Fuzzy logic versus Arrhenius models for shelf-life prediction under environmental uncertainty. Journal of Food Engineering, 367, 111892. https://doi.org/10.1016/j.jfoodeng.2023.111892

Wang, H., Zhang, Q., & Li, S. (2024). Dynamic shelf-life modelling for leafy vegetables under fluctuating temperature conditions. Postharvest Biology and Technology, 209, 112305.

Zhang, Y., Wang, Q., & Li, S. (2023). Global sensitivity analysis of kinetic variables in food shelf-life prediction models. Food Research International, 168, 112754.

Alarfaj, A. A., et al. (2024). Leveraging convolutional neural networks for disease detection in vegetables: A comprehensive review. MDPI, 14(10), 2231. https://www.mdpi.com/2073-4395/14/10/2231

Emegha, O., et al. (2025). Food security, food waste, and nutrition in Nigeria: Strategies, socio–cultural influences, and nutrition outcomes. ACTA TECHNICA CORVINIENSIS - Bulletin of Engineering, 18(4). https://acta.fih.upt.ro/pdf/2025-4/ACTA-2025-4-03.pdf

Ezeudoka, B. C., et al. (2026). Evaluation of household solid waste drivers, characteristics, management and its implications for sustainable environmental quality in Iwo, Nigeria. Frontiers in Sustainable Cities. https://doi.org/10.3389/frsc.2026.1812929

Fu, L., Sun, S., Li, R., & Wang, S. (2022). Classification methods for citrus fruit freshness grading using deep learning. Computers and Electronics in Agriculture, 193, 106679. https://doi.org/10.1016/j.compag.2021.106679

Hu, G., Zhang, K., & Wang, L. (2024). Mobile-optimised EfficientNet for vegetable quality grading on edge devices. Expert Systems with Applications, 238, 121872. https://doi.org/10.1016/j.eswa.2023.121872

Kumari, S. (2025). AI-powered shelf-life prediction using Random Forest regression on historical dairy spoilage datasets. Food Control, 157, 110185. https://doi.org/10.1016/j.foodcont.2024.110185

Leena, M. M. (2024). Predicting food shelf life: Advances in modelling and analytics. Trends in Food Science & Technology, 145, 104358. https://doi.org/10.1016/j.tifs.2024.104358

Nwafor, S. C. (2026). Effect of food waste on household dietary diversity in Nigeria. ResearchGate (Preprint). https://doi.org/10.21203/rs.3.rs-9474395/v1

Patel, R., Desai, A., & Shah, P. (2024). Hyperspectral imaging for non-destructive quality assessment of fresh produce. Journal of Food Science, 89(3), 1234–1248. https://doi.org/10.1111/1750-3841.16890

Sahu, A., Kumar, R., & Prasad, B. (2020). Edge IoT-based food quality monitoring system using Raspberry Pi. IEEE Internet of Things Journal, 7(8), 7338–7347.

Subari, A., et al. (2024). Fruit freshness detection using Android-based transfer learning MobileNetV2. ResearchGate.

https://www.researchgate.net/publication/396366328_Fruit_Freshness_Detection_Using_Android-Based_Transfer_Learning_MobileNetV2

Vasquez, R., Torres, J., & Mendoza, P. (2024). Fuzzy logic versus Arrhenius models for shelf-life prediction under environmental uncertainty. Journal of Food Engineering, 367, 111892. https://doi.org/10.1016/j.jfoodeng.2023.111892

Wang, Z., Li, M., & Zhao, R. (2025). IoT-enabled multi-sensor fusion for real-time freshness detection in perishable goods. Sensors and Actuators B: Chemical, 398, 134723.

Wibowo, S., Pratama, M., & Setiawan, A. (2022). Fuzzy inference system for shelf-life prediction of bakery products. Journal of Food Processing and Preservation, 46(8), e16812.

Yamamoto, K., Togami, T., & Yamaguchi, N. (2020). Surface defect detection of citrus fruits using deep CNN. Journal of Agricultural Engineering Research, 91(2), 45–53.

Zhang, Y., Wang, Q., & Li, S. (2023). Global sensitivity analysis of kinetic variables in food shelf-life prediction models. Food Research International, 168, 112754.